Tractable Problems in AI Security via Formal Methods

Quinn Dougherty
Forall R&D
quinn@for-all.dev

Max von Hippel
In an independent capacity
maxvh@hey.com

Abstract. Secure program synthesis is popping off in 2026 [1], [2], [3], which will be great for our overall cyber resilience. However, its not obvious that it will actually be applied to AI security in real life. To seize this opportunity, we need to map out the relevant layers in the current ML inference and training stack and figure out what widgets represent formal methods opportunities. Let us treat ML training and inference infrastructure with the seriousness we treat airplanes; and on the software side, that will mean at least some formal methods.

1 Executive Summary

1.1 How to read this document

The document has two halves. Section 2 walks the five layers of the ML training and inference stack — execution harness, software/ML framework, orchestration and cloud, firmware and low-level systems, hardware and physical supply chain — and, for each, sketches the status quo and how attackable it is. Section 3 is the payload: a list of concrete problems, each tagged by which layers of the stack it touches and whether it is an *enabler* (unbottlenecks a class of downstream work) or a *widget* (a scoped, shippable artifact). The website at tractable.for-all.dev mirrors the same content, with the layer/problem tagging exposed as a many-to-many filter.

1.2 What formal methods can and can't claim

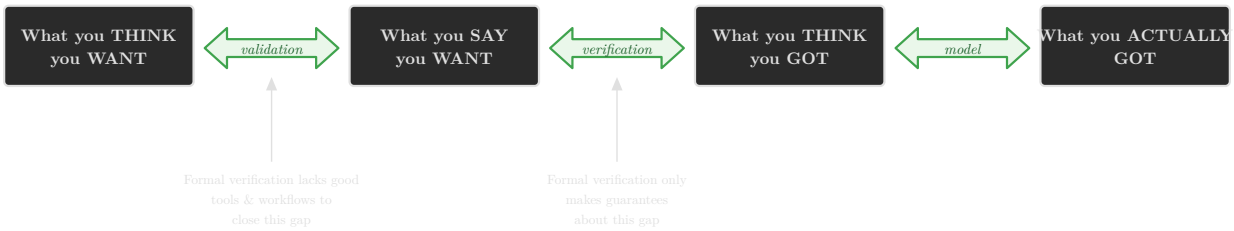

Figure 1: After Evan Miyazono. Formal verification only closes the middle gap; the elicitation gap on the left is out of scope for the proof itself, and the modeling gap on the right is only as good as the model of the system the proof is stated against. Both ends are where most real-world failures live.

1.3 Quick primer on formal methods

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

1.4 What we count as tractable

Three working principles narrow the problem list in Section 3.

1.4.1 Mechanism, not policy

Verification has the most leverage on the mechanisms that establish system integrity — microkernels, hypervisors, drivers, network protocols — and the least on the policies layered on top of them. Container scheduling on `Kubernetes` is the canonical example: a scheduler that places pods optimally produces a faster answer, not a more correct one, and verifying its placement decisions does not improve isolation between tenants if the underlying runtime is unsound. We accordingly scope the orchestration layer (Section 2.3) to its mechanism content — distributed-protocol correctness, IAM logic, network-fabric isolation, runtime confinement — and treat scheduler optimization as out of scope. The same line shows up at every layer: there is a verifiable mechanism underneath, and a policy or optimization above it whose correctness is downstream of the mechanism's.

1.4.2 Small API surfaces are reachable; large ones are not

Compare `NOVA`'s 16-hypercall interface to the `POSIX` surface that `Linux` exports, or to the API a `Docker` daemon exposes through its versions. The first is small enough to specify and prove against; the others balloon with each release and have no single coherent specification to target. Tractability tracks API size more reliably than it tracks codebase size — a small interface in front of a large implementation is a verification target, a large interface is not. This shapes the widget specs in Section 3 toward the smallest customer-relevant subset — the 10 or 25 functions a real workload actually uses — rather than full-coverage proofs of sprawling APIs.

1.4.3 Separable, language-agnostic specifications

A system's specification should be separable from its implementation, both technically (so a `Rust` rewrite of a `C++` component does not invalidate the proof effort) and legally (so an open spec can be referenced by implementations under stricter licenses). `NOVA` is the existence proof: the implementation is GPL-2 (Intel and TU Dresden) while the specification is licensed separately under Blue Rock [4]. The same separation is what would let a `Linux` retrofit happen without forcing every maintainer onto a single proof toolchain. We treat separability as a precondition: a widget that cannot factor cleanly into spec-and-implementation is one we cannot recommend, regardless of how shippable the implementation looks.

1.5 Why this is hard to fund

The honest difficulty: formal methods compete in a market where the unverified alternative is usually free. A startup considering whether to buy a verified hypervisor over `KVM`, or a verified container runtime over `runc`, is comparing a paid product against a zero-marginal-cost open-source incumbent that is good enough for the threat model the customer thinks they have. The ROI calculation is upside-down before the conversation starts. This is the central reason the formal-methods talent that exists today is concentrated in domains — avionics, defense, automotive —

where regulators force the comparison to be against a counterfactual incident rather than against the free alternative.

The document does not solve this problem, but it tries to make the right cases visible. For each tractable problem in Section 3, we name the threat model the verified component would close, the alternatives a buyer is implicitly comparing it against, and the lift to deliver a usable artifact rather than a research prototype. AI infrastructure is one of the few private-sector settings where the counterfactual incident is large enough to invert the calculation — model weight exfiltration, training-data poisoning, or container escape from an agentic workload are losses on the order of the model's training cost — and where the customers (frontier labs, regulated downstream deployers) have both the budget and the risk model to act on it. Making the business case requires actually writing it down. That is what this document is.

2 The ML Inference and Training Stack

The five layers below run top to bottom in the order an adversary walks the stack. Each section that follows opens on the layer's status quo and current attack surface, then surfaces the FM-shaped widgets that would close part of it. Scope follows Section 1.4 — infrastructure only; the model itself is out of frame.

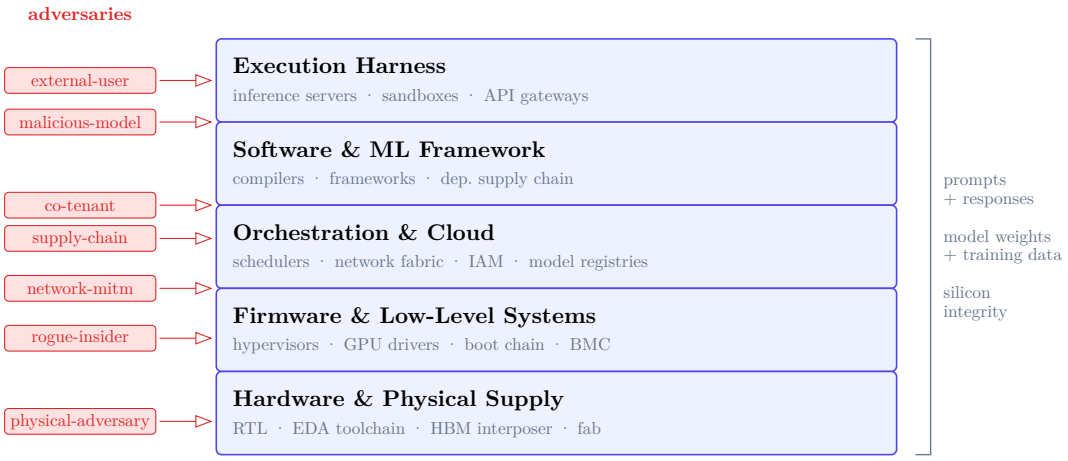

Figure 2: The five layers of the ML inference and training stack, with the adversary archetypes (left) that each layer invites and the assets (right) that pass through them. Adversaries are tagged on each layer below using the same labels; see Section 4 for full descriptions.

2.1 Execution Harness

Related problems: Adversarial Robustness of Formal Methods · Specification Elicitation and Validation · OCI Runtime Hardening · AI Control via Proof-Carrying Code · Edge Policy Verification · Weight Integrity and Kernel Supply Chain · Sampler Integrity and Determinism · Verified Input Parsers · Context Window Integrity Under Adversarial Length · Capability-Safe Tool Interfaces · Agent Loop Termination and Budget Enforcement · Audit Log Integrity and Session State Channels · Governance and Interpretability of Neuralese Proof Stacks · Formal Theory of Weird Machines in Agents

Invites: External User · Malicious Model · Co-tenant

The execution harness is the outermost software layer of an ML deployment: the wrapper that actually calls the model and handles its inputs and outputs. A user's prompt enters here, gets tokenized and batched, passes through the model, and returns as a completion. Because every

interaction transits this layer, it is simultaneously the easiest place to bolt on security controls and the place where a failure is most directly exploitable from the outside.

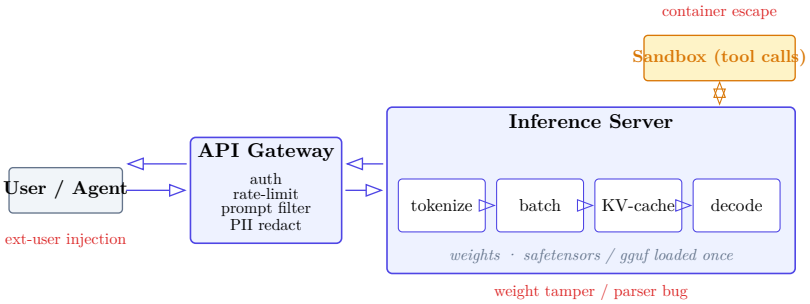

Figure 3: A typical request flow through the execution harness. The gateway makes policy decisions on the way in and the way back; the inference server holds the hot path; tool calls divert into a sandbox. The labelled red callouts mark the externally-reachable failure modes addressed later in Section 3.

Three components make up a typical execution harness, each with its own attack surface.

2.1.1 Inference Servers

An inference server (`vLLM`, Hugging Face `TGI`, Nvidia `Triton`) manages GPU memory, batches incoming requests for throughput, and exposes the model behind an HTTP or gRPC endpoint. The server is responsible for continuous batching, KV-cache management, and (increasingly) speculative decoding — all performance-sensitive paths written in a mix of Python and C++/`CUDA`.

The security-relevant surface here is large. The server deserializes model weights, often from formats like `SafeTensors` or `GGUF`, and the parsers for the latter have had their own history of memory-corruption bugs [5] (`SafeTensors`' own audit [6] found no critical flaws, but the `GGUF` loader in `llama.cpp` has seen repeated integer-overflow-to-heap-overflow patterns). It accepts user-controlled input (prompts, generation parameters, `LoRA` adapter names) that flows deep into the batching and memory-allocation logic. A malformed request that triggers an out-of-bounds KV-cache write, for instance, could corrupt another user's in-flight completion in a multi-tenant deployment. Formal methods work in this area has two legs: memory-safety of the batching and caching layers, and verified parsers across the several deserialization boundaries the server straddles (weights on the way in, request payloads on the hot path, and structured tool-call outputs on the way back). The second leg is the subject of Section 3.11; the tooling is mature and the gap is targeting, not capability.

Inference proxies like `LiteLLM` are also part of the dependency supply chain (Section 2.2.3) — a compromised package at this layer gives an attacker access to every API key and model endpoint the proxy routes.

2.1.2 Sandboxes

When an AI system can execute code — a "code interpreter" in a chatbot, a tool-use agent writing and running scripts — it needs a sandbox. In practice this means a containerized environment (often an OCI container or a microVM like `Firecracker`) with a constrained syscall profile and filesystem. The sandbox must prevent the AI-generated code from escaping to the host, exfiltrating data from other tenants, or persisting state across invocations in unintended ways.

The gap between "container" and "sandbox" is where real attacks live. Default container configurations are not security boundaries; they share a kernel with the host. A sandbox that allows `ptrace`, or that mounts the `Docker` socket, or that runs with `CAP_SYS_ADMIN`, is a sandbox in name only. The OCI runtime hardening problem (Section 3.4) addresses this directly: writing machine-checked specifications for what a sandbox policy must guarantee and verifying that a given runtime configuration meets them.

2.1.3 API Gateways

The API gateway sits at the public edge. It handles authentication, rate limiting, content filtering, prompt injection detection, and usage metering. In many deployments this is a conventional reverse proxy (`Envoy`, `Kong`) with AI-specific middleware bolted on: a classifier that scans prompts for injection attempts, a filter that redacts PII from completions, a logger that records interactions for abuse review.

The security challenge is that these filters are making semantic decisions about adversarial input using heuristics or secondary ML classifiers — a brittle arrangement. A prompt injection that evades the filter is not a bug in the filter's code but a failure of its classification boundary. Formal methods have limited traction on the classification problem itself, but they can verify the *plumbing*: that the gateway's policy engine correctly composes its rules, that a request flagged by the filter cannot reach the model through an alternate code path, and that rate-limiting state cannot be poisoned by concurrent requests. The value here is ensuring that when a policy says "block this," the infrastructure actually does.

2.2 Software and ML Framework Layer

Related problems: Adversarial Robustness of Formal Methods · Specification Elicitation and Validation · AI Control via Proof-Carrying Code · Weight Integrity and Kernel Supply Chain · Sampler Integrity and Determinism · Verified Input Parsers · Governance and Interpretability of Neuralese Proof Stacks

Invites: Supply Chain · Malicious Model · Rogue Insider

Between the orchestration infrastructure below and the execution harness above sits the software that actually defines and compiles a model: the frameworks researchers write against, the compilers that lower their code to GPU kernels, and the sprawling dependency graph that connects everything. This layer is where the mathematical definition of a neural network meets the reality of executable code. A compromise here is dangerous because it can alter what a model *computes* without changing what it appears to be — the weights, the architecture description, and the training logs all look clean while the compiled artifact does something else.

2.2.1 Compilers

An ML compiler takes a high-level model description and lowers it through several intermediate representations to GPU machine code. The typical path is something like `PyTorch` → `torch.compile` → `Triton` IR → `LLVM` IR → `PTX` → `SASS`, though the specifics vary: `TVM` uses its own `Relay`/`TIR` stack, `XLA` compiles through `HLO` and `StableHLO`, and `JAX` traces through `jaxpr` before hitting `XLA`. Each stage applies optimization passes — operator fusion, memory layout transforms, quantization, tiling — that rewrite the computation in ways the model author never sees directly.

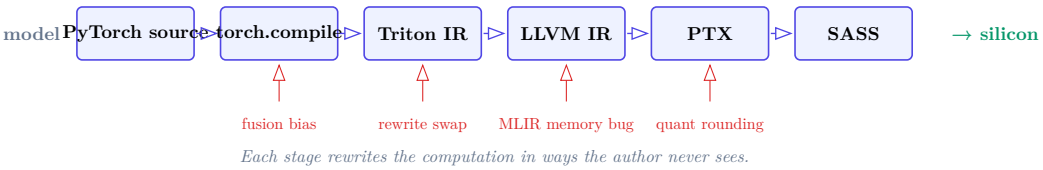

Figure 4: A typical `PyTorch` lowering path, with sample injection points (red) where a poisoned pass could alter what the silicon computes without changing what the source, weights, or training logs look like.

The attack surface is a modern instance of Thompson's "Trusting Trust." A poisoned compiler pass could inject a small additive bias into attention weights, swap an activation function under specific input conditions, or silently alter a quantization rounding mode. The result would be a model that behaves correctly on benchmarks but misbehaves on targeted inputs, with no trace of the modification in source code, model weights, or training logs. The compilation chain is long enough that auditing every stage by hand is not feasible.

This is not hypothetical. `LLVM`'s `MLIR` infrastructure — shared by `TVM`, `XLA`, and `Triton` — had multiple memory-management vulnerabilities in 2023 [7] where a crafted `MLIR` file could trigger crashes or worse. More telling is a 2025 `XLA`:TPU miscompilation in the approximate top-k operation that returned silently wrong results [8] for certain batch sizes, affecting production models. The compiler produced valid-looking output; it was just computing the wrong function. No error was raised. This is the failure mode that formal methods are built to prevent.

The formal methods community has begun to engage. Bhat et al. [9] formalized a core calculus for SSA-based IRs in `Lean` and verified peephole rewrites across three use cases: `LLVM` bitvector rewrites, structured control flow, and fully homomorphic encryption. Fehr et al. [10] defined semantics for five `MLIR` dialects, built three verification tools, and found five miscompilation bugs in upstream `MLIR` in the process. Both cover individual rewrites and local properties, not end-to-end compilation correctness. The gap between "verified peephole rewrites" and "verified compiler" is the same gap `CompCert` [11] closed for C — and closing it for even one ML compilation path (say, `StableHLO` to `PTX` for a fixed set of operations) would be a major result.

A practical intermediate target: verified compilation of attention kernels. Attention is the computational core of transformer inference, and it is also the operation most aggressively optimized by ML compilers (`FlashAttention`, `PagedAttention`, various fused variants). Proving that the compiled kernel preserves the semantics of the source-level attention specification, up to documented floating-point tolerances, would cover the highest-value target in the compilation chain.

2.2.2 Frameworks

`PyTorch`, `JAX`, and `TensorFlow` are the environments where models are defined, trained, and exported. They provide automatic differentiation, GPU dispatch, a zoo of layer implementations, and serialization formats for saving and loading model weights. Each is a million-line codebase with native extensions, JIT compilers, and deep operating system integration, but the highest-impact vulnerability class is mundane: deserialization.

`PyTorch`'s default serialization uses Python's `pickle` protocol, which can embed arbitrary executable code in a saved file. Loading an untrusted `.pt` file is functionally equivalent to running `eval()` on attacker-controlled input. This was understood in principle for years, but [12] demonstrated

that even the `weights_only=True` safeguard — `PyTorch`'s own recommended mitigation — could be bypassed, restoring full remote code execution. `Hugging Face` uses `PickleScan` to screen uploaded models, but `JFrog` found three zero-day bypasses in the scanner itself [13]: one corrupts magic-number detection so the scanner halts with an "invalid magic number" error while `torch.load()` still loads the payload; a second exploits a file-extension mismatch that skips scanning entirely; and a third uses submodule imports to downgrade the severity of dangerous globals from blocked to merely flagged. In a separate line of attack, `ReversingLabs`' nullifAI technique [14] hid working reverse shells in `PyTorch` models by 7z-compressing the `pickle` payload so the scanner's decompression failed, even as the malicious code ran first. `TensorFlow` has its own variant: `Keras` Lambda layers embed arbitrary Python in model definitions, and [15] showed that the `safe_mode` fix could be bypassed via a downgrade attack when loading older checkpoints.

The `SafeTensors` format, developed by `Hugging Face`, stores only tensor data and metadata with no code execution path. An independent security audit [6] found no critical flaws. This is the right direction, but the safety argument rests on an audit, not a proof. A serialization format with a machine-checked parser — in the style of `EverParse` or similar verified parsers — would close the vulnerability class permanently rather than playing whack-a-mole with scanner bypasses.

Beyond serialization, the frameworks expose attack surface through JIT compilation and custom operators. `PyTorch`'s `torch.utils.cpp_extension.load()` compiles C++/`CUDA` code into `/tmp` and dynamically loads the resulting shared library. Any process that can influence the source or the cached `.so` gets native code execution. `TensorFlow` has accumulated roughly 435 CVEs across 44 CWE categories [16], many in parsing and memory management of its graph representation. Formal verification of core tensor operations — proving that `softmax`, `layernorm`, or `matmul` produce results consistent with their mathematical definitions across all compilation backends — remains open territory.

2.2.3 Dependency Supply Chain

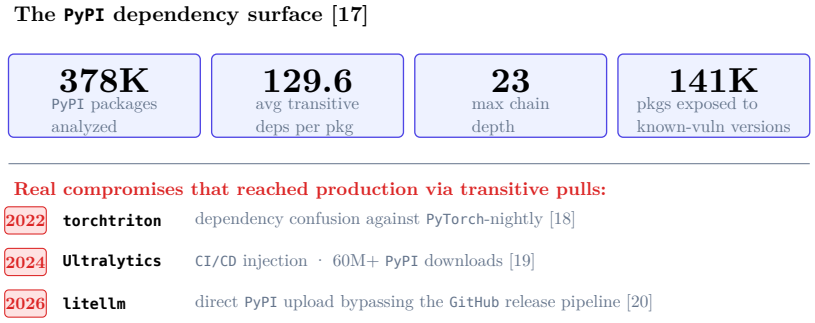

Figure 5: Top: ecosystem-level statistics on the transitive-dependency surface from [17]. Bottom: three real compromises that reached production ML stacks via transitive pulls in 2022–2026.

A typical ML project does not just depend on `PyTorch` or `TensorFlow`. It depends on a graph of packages — data loaders, tokenizers, experiment trackers, serving utilities, `CUDA` bindings — that pull in their own transitive dependencies. Mahon et al. [17] analyzed 378,573 `PyPI` packages and found that the average package has 129.6 transitive dependencies spanning a dependency chain up to 23 levels deep. A single CVE in `urllib3` created guaranteed exposure in 1,906 downstream packages. The ML ecosystem sits on top of all of this and adds its own layer of domain-specific packages with fast release cycles and small maintainer teams.

Supply chain attacks against this graph are not theoretical. In December 2022, an attacker uploaded a malicious `torchtriton` package to `PyPI` with the same name as `PyTorch`'s internal dependency [18]. Because `pip` resolves the public `PyPI` index before private indices by default, `PyTorch`-nightly users received the attacker's version, which exfiltrated hostnames, environment variables, and password files. In December 2024, the `Ultralytics` YOLO library (60+ million `PyPI` downloads) was compromised through a `GitHub Actions` script injection: the attacker crafted PR branch names that injected code into the CI/CD pipeline, and the published package bundled a cryptominer [19]. In 2026, the legitimate `litellm` package — an inference proxy pulled as a transitive dependency by many ML stacks — was compromised by a direct upload to `PyPI` that bypassed the project's `GitHub` release pipeline entirely [20]. The injected payload harvested SSH keys and cloud credentials and attempted lateral movement across `Kubernetes` clusters. The attacks are moving up the dependency graph, where one package reaches thousands of dependents.

The defenses that exist are largely unsigned and unverified. `SLSA` (Supply-chain Levels for Software Artifacts) [21] defines four levels of provenance assurance, from basic metadata (L1) through hermetic reproducible builds (L4). Most ML toolchains operate at L0 or L1 — packages are uploaded by individual maintainers from their laptops with no build provenance, no signed attestation, and no reproducibility guarantee. `Sigstore` [22] offers keyless artifact signing tied to OIDC identities, which would at least bind a package version to a specific CI run. But even with signed provenance, the resolver itself is a trust boundary: `pip`'s dependency resolution algorithm is complex, underspecified, and has produced dependency confusion vulnerabilities by design.

Formal methods can contribute at two levels. First, at the resolver level: proving that a dependency resolution algorithm correctly implements its specification and cannot be tricked by namespace collisions, version range manipulation, or index priority ordering. This is a bounded, well-defined verification target. Second, at the build level: verified reproducible builds where the mapping from source to artifact is proven deterministic, so that any party can independently verify that a published package corresponds exactly to its claimed source. Achieving `SLSA` L4 with machine-checked guarantees — rather than relying on build system configuration being correct — would make supply chain substitution attacks structurally impossible rather than merely detectable.

2.3 Orchestration and Cloud Layer

Related problems: Specification Elicitation and Validation · OCI Runtime Hardening · AI Control via Proof-Carrying Code · Edge Policy Verification · Scheduler Integrity and Co-Tenancy Isolation · Verified Network Tap · Audit Log Integrity and Session State Channels · Governance and Interpretability of Neuralese Proof Stacks · Formal Theory of Weird Machines in Agents

Invites: Co-tenant · Rogue Insider · Network MITM · Supply Chain

The orchestration layer manages the compute resources that ML workloads run on: scheduling jobs across GPU clusters, routing traffic between them, and controlling who can access what. It sits below the frameworks and above the firmware — a layer of distributed systems software that most ML engineers interact with only through configuration files, but whose security properties determine whether isolation between tenants, jobs, and data pipelines actually holds. Following Section 1.4.1 we treat this layer as a stack of mechanisms (runtime confinement, distributed-protocol correctness, network-fabric isolation, IAM logic) and not as a question about scheduler optimization; placement and load-balancing decisions are downstream of these mechanisms and out of scope.

2.3.1 Cluster Orchestration

GPU training and inference jobs run inside containers managed by `Kubernetes`, `Slurm`, or `Ray`. The container is the unit of isolation: it is supposed to confine a workload to its own filesystem, network namespace, and device access. In practice, GPU workloads erode this confinement. NVIDIA's container toolkit — the standard mechanism for exposing GPUs inside containers — had a critical TOCTOU vulnerability [23] that allowed a malicious container image to escape to the host, affecting 33% of cloud environments per Wiz's estimate. The underlying container runtime, `runc`, had its own escape [24] via leaked file descriptors. These are not exotic attacks; they are the kind of bugs that container runtimes accumulate routinely, and they interact badly with the privileged device access that GPU workloads require. The OCI runtime hardening problem (Section 3.4) addresses the gap between what orchestrators *assume* containers guarantee and what the runtimes actually enforce.

`Slurm`, the dominant scheduler for HPC and large training runs, has a worse security record than most ML engineers realize. It has historically run its daemons as root, and its authentication rests on `MUNGE` — a shared-secret credential system that was never designed for adversarial multi-tenant environments. [25] demonstrated this: attackers could bypass `MUNGE`'s hash-based message integrity protections in the `slurmd` process to replay root-level authentication tokens, gaining privileged access to compute nodes over the network. Earlier vulnerabilities were blunter — [26] let an unprivileged user send data to arbitrary Unix sockets on the host as root through `Slurm`'s `PMI2`/`PMIx` RPC handler, and [27] allowed privilege escalation to root via `SPANK` environment variable injection during Prolog/Epilog execution. `Slurm` clusters at national labs and cloud GPU providers routinely run versions months or years behind patches, because upgrading the scheduler means draining the entire cluster.

`Ray`'s security posture is, by its maintainers' admission, a design choice rather than an oversight. Until recently, `Ray` had no authentication on its Jobs API or Dashboard — any process that could reach the network port could submit arbitrary code for execution on the cluster. Anyscale's position was that `Ray` should only run in isolated networks and act upon trusted code. The result was predictable: the ShadowRay campaign [28] compromised hundreds of thousands of exposed `Ray` clusters for cryptocurrency mining, data exfiltration, and eventually self-propagating botnets. Attackers extracted production database credentials, cloud environment tokens, and model weights from compromised AI workloads. The vulnerability remained unpatched for over two years because the vendor classified it as intended behavior.

`Kubernetes` introduces its own GPU-specific attack surface through device plugins. The NVIDIA device plugin runs as a privileged `DaemonSet` with access to the host's `/var/lib/kubelet/device-plugins` socket and device files. A compromised plugin can register fake GPU devices, intercept `kubelet` communications, or escalate to full node compromise. `Kubernetes` also lacks native GPU resource throttling — unlike CPU and memory, there are no `cgroup` controls for GPU compute time, so a malicious pod can monopolize GPU cycles and starve other tenants' inference workloads with no scheduler-level mitigation. In multi-tenant clusters, GPU memory is not isolated by default; processes on the same GPU can read each other's memory regions unless `MIG` (Multi-Instance GPU) or `MPS` partitioning is explicitly configured, and even then the isolation guarantees are weaker than what hypervisors provide for CPU and RAM.

2.3.2 Network Fabric

Large training runs communicate over `InfiniBand` or `RoCE` interconnects using RDMA — remote direct memory access that bypasses the kernel to move data between GPU memory regions at line rate. This is a performance architecture, not a security architecture. RDMA's kernel bypass means that the standard OS-level network monitoring and access control stack is not in the path. `InfiniBand` partition keys (`P_Keys`) provide coarse tenant isolation at the hardware level, but verifying that traffic isolation policies are correctly enforced across a fabric of thousands of ports is a manual, configuration-driven process with no runtime audit trail. If you want to verify what is actually on the wire between nodes in a GPU cluster — not what the SDN controller *says* is on the wire — you need an independent observation point, which is the problem the verified network tap (Section 3.8) is designed to solve.

The SDN controllers that manage GPU cluster fabrics are themselves a concentrated target. An SDN controller is a single logical authority over all traffic routing decisions in the network; compromise it and you can redirect, mirror, or drop any flow. The `OpenFlow` protocol, which most SDN deployments use for controller-to-switch communication, did not require authentication of switches in early versions, and even post-1.2 versions with TLS support are frequently deployed without it. An attacker who gains access to the control plane can add a second malicious controller (a feature available in `OpenFlow` 1.2+) and persistently reroute traffic without touching the data plane hardware. The centralization that makes SDN manageable is exactly what makes it a single point of failure for traffic integrity.

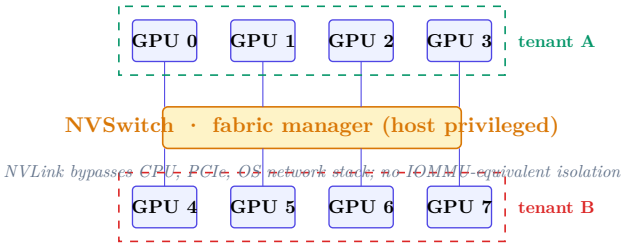

Figure 6: `NVLink`/`NVSwitch` topology inside a typical eight-GPU node. The fabric manager that programs `NVSwitch` routing tables runs as a privileged host process; the resulting tenant boundary is software-configured rather than hardware-enforced, and the traffic that crosses it never touches the CPU, the PCIe bus, or the OS network stack.

Within a single node, GPUs communicate over `NVLink` and `NVSwitch` at bandwidths up to 900 GB/s — bypassing the host CPU, PCIe bus, and OS network stack. NVIDIA's fabric manager controls `NVSwitch` routing tables and restricts applications to designated address ranges, but this is a software-configured trust boundary, not a hardware-enforced one. There is no equivalent of IOMMU page-table isolation for `NVLink` traffic; the protection rests on correct configuration of the fabric manager, which runs as a privileged host process. A compromised fabric manager, or a bug in NVSwitch routing table setup, could allow one GPU's process to read or write another's memory region across the switch. For multi-tenant GPU servers using `NVLink`-connected GPUs — the standard configuration in `DGX` systems and cloud GPU instances — this is a lateral movement path that never touches the network and never appears in host OS logs.

Even when interconnect traffic is encrypted, traffic analysis remains viable. The volume and timing of gradient synchronization traffic are highly regular and predictable — all-reduce operations

produce characteristic burst patterns that correlate with batch boundaries, model architecture, and even convergence state. An observer on the fabric (or with access to switch port counters) can infer training progress, detect when checkpointing occurs, and distinguish between different model architectures without decrypting a single packet.

2.3.3 Distributed Systems

Distributed training coordinates gradient synchronization across hundreds or thousands of GPUs using collective operations — all-reduce, all-gather, reduce-scatter — implemented by libraries like `NCCL` and `Gloo`. These protocols run over the RDMA fabric with no authentication or encryption; any process that can reach the network can inject or modify gradient data in transit. In federated and multi-node settings, this is the mechanism by which model poisoning attacks operate: a compromised node contributes manipulated gradients that shift the model's behavior while remaining within the statistical noise of normal training variance. The protocol's correctness properties — that all-reduce actually computes the sum of all participants' contributions, that no participant can observe another's individual gradient — are distributed-system invariants that formal methods can specify and verify. Proof-carrying code (Section 3.5) is one path to making these guarantees checkable at deployment time rather than assumed.

Checkpoint integrity is the other open wound. Large training runs write distributed checkpoints — sharded model state spread across dozens or hundreds of files on networked storage — every few thousand steps. These checkpoints are the recovery mechanism when nodes fail, which happens constantly at scale. They are also the artifact that gets promoted to production: the "final model" is just the last checkpoint that passed evaluation. Yet checkpoints are typically stored as serialized tensors with no cryptographic signature, no content-addressable hash chain linking them to the training run that produced them, and no tamper-evident log of which processes wrote which shards. An attacker with write access to the shared filesystem — or to the object store behind it — can modify checkpoint shards to inject backdoor behavior, and the training job will resume from the poisoned state without any indication that the checkpoint was altered. The same exposure applies when checkpoints are copied between clusters, uploaded to model registries, or handed off from a training team to a deployment team. The integrity gap between "this file exists on S3" and "this file is the authentic output of training run X at step Y" is currently bridged by convention, not cryptography.

Data pipeline integrity compounds the problem. Training data flows through distributed storage systems — `HDFS`, cloud object stores, data lakes — through preprocessing jobs, shuffling, and batching before it reaches the GPUs. Each stage is a point where data can be modified, filtered, or augmented by an attacker with access to the storage layer. Unlike gradient poisoning, which requires a compromised training node, data poisoning can happen long before training begins, in ETL jobs or data labeling pipelines that are often run with broader permissions and less monitoring than the training jobs themselves.

2.3.4 Identity and Access Management

The IAM layer controls which principals — human users, CI pipelines, serving infrastructure, and increasingly autonomous agents — can read model weights, write to training data stores, modify deployment configurations, or invoke models. The failure mode is over-permissioning: ML pipelines tend to accumulate broad service account credentials because the alternative is debugging

permission errors during a training run. In 2025, Sysdig's threat-research team documented an attack chain [29] where compromised credentials reached cloud administrator privileges in eight minutes across 19 distinct AWS principals — LLM-assisted reconnaissance followed by Lambda-based privilege escalation, with invocation logging disabled along the way. When the principal is an AI agent operating with long-lived API tokens, the IAM boundary *is* the control boundary. A token compromise gives the attacker everything the agent could do, with access patterns indistinguishable from legitimate automation. Proof-carrying code (Section 3.5) offers a structural alternative: rather than trusting that IAM policies are correctly configured and that tokens are uncompromised, require that actions carry machine-checkable proofs of authorization.

Model registries are an under-secured chokepoint. A model registry — Hugging Face Hub, a private MLflow instance, a cloud model catalog — is where trained models are stored, versioned, and promoted to production. Whoever can push a new version to the registry controls what runs in production. The PoisonGPT demonstration [30] showed that a surgically modified open-source model could be uploaded to Hugging Face and spread targeted misinformation while retaining normal benchmark performance. ReversingLabs found malicious code embedded in pickle-serialized models on Hugging Face that evaded the platform's safety scanning [14]. Model confusion attacks — the AI supply-chain analog of dependency confusion — exploit namespace reuse in registries to trick downstream pipelines into pulling attacker-controlled models. Access control on model registries is often weaker than on code repositories: push permissions are granted to broad service accounts, there is no equivalent of signed commits or required code review, and the model artifact format (often pickle or safetensors) has no built-in integrity or provenance chain.

Secret sprawl is the background condition that makes all of the above worse. ML workflows generate and consume credentials at every stage: cloud storage keys in training scripts, Weights & Biases tokens in experiment configs, Hugging Face tokens in model download scripts, database credentials in data pipeline notebooks. GitGuardian's 2025 report [31] found 23.8 million secrets leaked in public GitHub repositories, a 25% increase over the prior year — and ML repositories are disproportionately affected because the notebook-driven, experiment-oriented workflow encourages hardcoding credentials for quick iteration. These secrets leak into container images (baked into layers during build), into training logs (printed during debugging and never scrubbed), and into checkpoint metadata (serialized alongside model state). When an agent operates autonomously — writing code, calling APIs, launching training runs — it accumulates credentials in its context window and working files with no human in the loop to notice. The agent-to-agent delegation problem makes this worse: when one agent delegates a subtask to another, what credentials does the delegate inherit, and who revokes them when the task is done? Current IAM systems have no concept of transitive, time-bounded delegation for non-human principals.

2.4 Firmware and Low-Level Systems

Related problems: GPU Drivers for Verified Kernels · Weight Integrity and Kernel Supply Chain
Invites: Co-tenant · Rogue Insider · Supply Chain

Below the orchestration layer and above the silicon sits the firmware: hypervisors, device drivers, and boot chains. Code here runs at the highest privilege levels on both CPU and GPU. A bug is not a container escape — it is a host compromise, often with no log entry at all.

2.4.1 Microkernels and Hypervisors

Multi-tenant GPU isolation today relies on hypervisors. KVM's trusted computing base is roughly ten million lines of code; Xen is smaller but still far too large for exhaustive verification. Both have had VM-escape vulnerabilities — Google Project Zero's 2021 KVM breakout via AMD SVM nested virtualization [32] is a representative example — and the PCI passthrough path that GPU workloads require widens the attack surface further, since IOMMU misconfigurations or missing PCI Access Control Services (ACS) can allow peer-to-peer DMA between devices assigned to different tenants. NVIDIA's Multi-Instance GPU (MIG) partitioning provides hardware-level memory isolation within a single GPU, but the partitioning is managed by the host driver, which runs inside the hypervisor's TCB. seL4, the only general-purpose microkernel with a complete functional-correctness proof, has no GPU driver support at all. Its new device driver framework (sDDF) handles network and block devices, but nothing with the complexity of a modern GPU. NOVA [4] is the other candidate worth naming here: it is open-source (GPL-2) with an open-source proof, and though the verification is not as far along as seL4′s, the portions that are complete already account for concurrency and weak memory — both of which fundamentally change the verification challenge rather than merely extending it. Closing the GPU gap — giving either kernel a verified GPU driver — is the subject of Section 3.3.

Even where MIG is deployed, GPU memory is not always scrubbed between context switches. Trail of Bits demonstrated this with LeftoverLocals [33]: on AMD, Apple, and Qualcomm GPUs, a co-resident process could read local memory left behind by a previous kernel, recovering roughly 5.5 MB per GPU invocation — enough to reconstruct an LLM's token-by-token output with high fidelity. NVIDIA hardware was not affected in that specific case, but the deeper issue is architectural. GPUs were designed for throughput, not isolation, and the memory hierarchy reflects that priority. Shared L2 caches, shared interconnects, and performance counters that leak timing information all create side channels across tenant boundaries. The NVBleed attack [34] showed that contention on NVLink interconnects between GPUs lets an attacker fingerprint which deep-learning model a co-tenant is running with 97.8% accuracy, and the attack works across VM boundaries on Google Cloud — even after NVIDIA patched performance-counter access, the timing channel alone still yields F1 scores above 83%.

Confidential VMs compound the problem. CPU-side TEEs like AMD SEV-SNP encrypt guest memory so the hypervisor cannot read it, but extending that guarantee to a GPU is an open engineering problem. SEV-SNP requires the IOMMU in non-passthrough mode to prevent peripherals from reaching encrypted memory, which directly conflicts with the PCIe passthrough that GPU workloads need. AMD's SEV-TIO extension [35] is meant to bridge this gap using PCI-SIG's TDISP protocol, but it is not yet widely deployed, and the attestation story for a GPU behind a TDISP-secured link is still being worked out. In the meantime, any "confidential AI" deployment that claims SEV-SNP protection while using GPU passthrough has a gap in its threat model that the marketing materials do not mention.

2.4.2 Device Drivers and Runtimes

The GPU driver is the most privileged code that touches the accelerator. NVIDIA's proprietary CUDA driver stack, AMD's ROCm, and Intel's oneAPI are all closed-source, kernel-mode codebases that handle memory mapping, command submission, and context switching for every GPU workload on the machine. NVIDIA ships quarterly security bulletins; 2024 alone included [36] (privilege

escalation in the display driver) and [37] (out-of-bounds read leading to code execution), plus nine vulnerabilities in the `CUDA` toolkit found by Palo Alto's Unit 42 [38]. These are not exotic attacks — they are standard memory-safety bugs in C code running at ring 0. The driver is a single point of compromise: an attacker who controls it can read any tenant's GPU memory, modify in-flight computations, or pivot to the host kernel. Because the driver source is proprietary, the only available mitigations are patching and hoping. A verified, open driver for at least the GPU command-submission path would change the calculus (Section 3.3).

A compromised or buggy GPU driver also opens the door to DMA attacks. The GPU sits on the PCIe bus and performs DMA to host memory; the IOMMU is supposed to restrict which physical pages the device can touch, but if the driver programs the IOMMU mappings incorrectly — or if an attacker can influence them — the GPU becomes a tool for reading or writing arbitrary host memory. [39] demonstrated exactly this on NVIDIA's Jetson platform, where a PCIe DMA attack bypassed secure boot. The IOMMU is a necessary defense, but it is only as good as the code that configures it, and that code is part of the same proprietary driver stack.

The `CUDA` compatibility layer adds its own surface area. NVIDIA maintains forward and backward compatibility across major `CUDA` versions, which means the driver must support multiple ABIs simultaneously and preserve behavior across a sprawling matrix of toolkit-to-driver version combinations. Operators running multi-framework training pipelines frequently pin older driver versions to avoid breaking their stack, which means known-patched vulnerabilities persist in production for months or years. The compatibility shims themselves are additional code paths that rarely get the same scrutiny as the hot path, and any bug in ABI translation is a bug at ring 0.

On the open-source side, NVIDIA released its kernel-mode GPU modules as open source in 2022, and the `Mesa` project's `NVK` driver now provides `Vulkan` support for `Kepler` through `Blackwell`. These are steps forward for auditability — community review can find classes of bugs that internal QA misses. But neither effort covers the full compute path that ML workloads use: the user-mode `CUDA` runtime, the compiler backend, and the firmware blobs that run on the GPU's internal microcontrollers all remain closed. An open kernel module with a closed firmware blob is better than nothing, but it is not a verifiable system. The gap between "source available" and "formally verified" is where the interesting work lies.

Worth naming what the open problem is and is not. Driver verification as a research methodology — separation-logic proofs of a driver against an abstract hardware model — is a demonstrated pattern [40], [41], [42]. The unsolved half is the abstract hardware model itself. Vendors do not ship machine-checkable specifications of their devices; the most accurate public modeling effort, Peter Sewell's REMS Group under a long-running ARM collaboration [43], is scoped to the CPU ISA and does not cover the MMU, IOMMU, or anything resembling a GPU command processor. The pivot from such a model to a software proof is a further gap; sketches exist [42] but no one has closed it at scale. The tractable problem at Section 3.3 is accordingly as much about producing a formal model of a GPU command-submission interface as it is about proving a driver against one.

2.4.3 Boot Integrity and Firmware Supply Chain

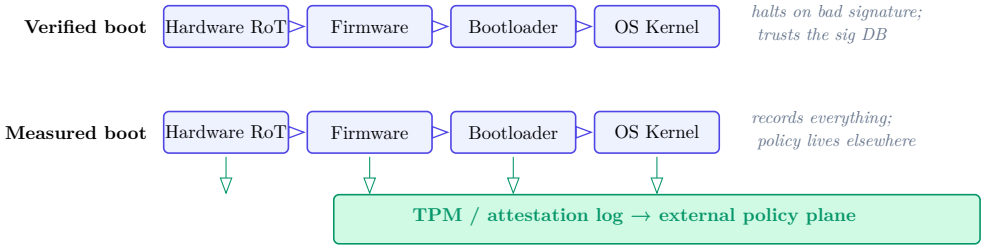

Figure 7: Two strategies for boot-chain integrity. *Verified boot* halts on a bad signature at each stage but trusts whatever lives in the signature database. *Measured boot* records hashes into a TPM and defers enforcement to an external policy plane that can release secrets, quarantine, or hard-reset based on attestation.

None of the isolation above matters if the firmware itself has been tampered with. Secure Boot and measured boot chains establish trust from the hardware root of trust (RoT) through each firmware stage to the OS kernel: each stage cryptographically verifies the next before handing off execution. NVIDIA's H100 extends this model to the GPU, with a per-device ECC keypair, on-die RoT, and a measured boot sequence that produces an attestation report — a signed manifest of every firmware component loaded. Combined with CPU-side TEEs (Intel `TDX`, AMD `SEV-SNP`), this enables composite remote attestation: a verifier can check that both CPU and GPU booted clean firmware before releasing model weights or training data to a node.

There is a real architectural choice here, not just a terminology one, between enforcing boot locally via signature check (verified boot) and enforcing it externally via an attestation-gated control plane (measured boot). Verified boot halts on bad signatures, but it trusts whatever is in the signature database, and that database has been bypassed in the wild: the BlackLotus bootkit (2023) exploited [44] to defeat UEFI Secure Boot on fully patched Windows systems by bringing its own copy of a legitimately signed but vulnerable boot manager. Measured boot does not prevent anything from loading — the TPM faithfully records whatever hash is presented, clean or not — but in exchange it is more compositional: the hardware only measures, and policy (what to do with a given measurement) lives outside the boot path, in an operator-owned plane that can release secrets only to nodes that pass attestation, hard reset machines that fail it, or quarantine them from the management network for analysis. `NOVA` deliberately went this direction [4], on the argument that post-facto measurability with an external control plane is a better composition point than bundling enforcement into the boot ROM. Either story depends on continuous attestation and a fresh revocation list; a stale verifier leaves a tampered node operating undetected between checks.

The Baseboard Management Controller (BMC) is a separate, quieter threat. Every server in a training cluster has a BMC — a small system-on-chip running its own OS (often Linux), connected to its own network interface, with out-of-band access to the host's power, console, and firmware update mechanisms. BMCs are managed via `IPMI` or `Redfish`, and they are rarely patched with the same urgency as the host OS. In 2023, Binarly disclosed seven vulnerabilities in Supermicro BMC firmware [45] that gave unauthenticated attackers root access to the BMC. From there, an attacker can read cleartext credentials off the BMC filesystem, flash malicious UEFI firmware to the host, or pivot to every other BMC on the management VLAN. The persistence is the real

problem: a BMC implant survives host OS reinstalls, disk wipes, and even GPU firmware updates, because it lives on a separate flash chip that the host never touches.

Firmware signing for GPUs is better than it was — NVIDIA's secure firmware update mechanism verifies digital signatures and enforces version anti-rollback — but the trust anchor is only as strong as the key management around it. The 2022 LAPSUS$ breach of NVIDIA [46] resulted in the theft of two code-signing certificates. Although the certificates were expired, Windows still accepts expired certificates for kernel drivers, and malware signed with the stolen keys appeared in the wild within a day. The broader lesson: a single key compromise turns the entire firmware-signing infrastructure from a defense into a distribution channel. What formal methods can contribute here is verifying the attestation protocol itself: proving that the chain of measurements is unforgeable, that a compromised node cannot replay a clean attestation, that revocation logic has no time-of-check/time-of-use gaps, and that the BMC's update path cannot be used to write outside its intended flash region. The machinery exists, but deploying it across a 10,000-GPU training cluster with continuous attestation, key rotation, and revocation checking is an engineering problem that remains largely unsolved at scale.

2.5 Hardware and Physical Supply Chain

Related problems: GPU Drivers for Verified Kernels · Verified Network Tap
Invites: Supply Chain · Physical Adversary · Rogue Insider

At the bottom of the stack is the silicon itself: the GPU, TPU, and NPU dies that run every training and inference workload, and the global supply chain that produces them. Compromise here is durable: a hardware trojan survives every software update, every OS reinstall, every firmware reflash. It is also opaque: you cannot read out the RTL of a fabricated chip to check what it does.

2.5.1 Chip Architecture

The RTL design of a modern AI accelerator is proprietary. NVIDIA does not publish the register-transfer-level description of its GPU compute cores; neither do Google (TPUs), AMD, or any other vendor. This means the formal methods community cannot verify what the silicon actually computes. We take it on trust that an H100′s tensor cores implement matrix multiplication correctly, that its memory controllers enforce isolation between MIG partitions, and that no undocumented debug interfaces exist. Classen et al. [47] demonstrated that a hardware trojan implanted in a Xilinx `Vitis AI` DPU could backdoor a traffic-sign recognition model by replacing just 30 of 43,000 parameters at the accelerator level — expanding the circuit by 0.24% with zero runtime overhead, invisible to all software-side defenses. The attack needed no cooperation from the model or the training pipeline; the accelerator alone was sufficient.

The design tools themselves are part of the attack surface. The chip industry flows through a handful of EDA vendors — primarily `Synopsys` and `Cadence` — whose toolchains are millions of lines of proprietary code that no customer audits. Basu et al. demonstrated in their "CAD-Base" work [48] that every stage of the EDA pipeline — high-level synthesis, logic synthesis, place-and-route, verification — is a viable insertion point for hardware trojans. The place-and-route attack is nasty: the tool can emit one netlist for verification and a different netlist for tapeout, and existing equivalence-checking flows never compare the two. A malicious logic synthesis tool can exploit underspecified finite-state machines to insert extra transitions triggered by rare conditions, leading the circuit into attacker-controlled states. These are not hypothetical proof-of-concept

attacks against toy designs; they target the actual tool interfaces that every fabricated chip passes through. Open-source EDA toolchains (`Yosys`, `OpenROAD`) exist but are not used for leading-edge AI accelerator design, so for now the industry trusts its toolchain vendors implicitly.

Beyond the design tools, modern SoCs integrate dozens of third-party IP cores — licensed blocks for PCIe controllers, memory interfaces, interconnect fabrics, security modules — that the chip designer incorporates without access to the underlying RTL. A third-party IP vendor can embed a hardware trojan that activates post-fabrication, leaks data through a covert channel, or degrades reliability on a trigger condition. The SoC integrator's only recourse is black-box testing and whatever contractual guarantees the IP license provides, neither of which catch a well-designed trojan. Formal verification of third-party IP at the pre-silicon stage is an active research area, but it requires a golden specification of correct behavior — which is exactly what you don't have when the IP is a licensed black box.

Hardware formal verification *is* used in industry, but its coverage is narrower than outsiders might expect. Intel adopted formal methods in earnest after the 1994 Pentium FDIV bug (a $475 million recall) [49]; by the Nehalem microarchitecture in 2008, formal verification was the primary validation method for floating-point and other arithmetic units. ARM's `ISA-Formal` framework [50] uses bounded model checking in `JasperGold` to verify that RTL implementations conform to the instruction set specification — but bounded model checking only proves correctness up to a fixed sequence length, not for arbitrary execution traces, unless you can find the right invariants to lift it to an unbounded proof. In practice, formal verification at major vendors covers specific high-value blocks (FPUs, cache coherence protocols, security state machines) while the rest of the design relies on simulation-based testing. Nobody is formally verifying an entire GPU.

RISC-V changes part of this picture. The ISA specification is open, and several open-source RTL implementations exist (`BOOM`, `Rocket`, `CVA6`) with corresponding formal verification frameworks — `YosysHQ`'s `riscv-formal` [51] can check that an implementation conforms to the RISC-V spec using model checking, and the OpenHW Group's `CORE-V-VERIF` [52] provides industrial-grade verification for the CV32 and CV64 families. MIT's Riscy processors, built on the Coq-embedded `Kami` DSL [53], are formally verified open-source RISC-V cores. But open ISA is not open silicon. These designs still get fabricated at foundries through proprietary EDA toolchains and opaque manufacturing processes. An open RTL that you can verify pre-silicon is a real improvement over a proprietary RTL that you can't — but it does not close the gap between the verified netlist and the actual transistors on the die. FPGAs remain the one place where the deployer controls the entire path from RTL to running hardware, which is why Section 3.8 proposes FPGA taps as a verification target.

On the GPU side, the architecture determines what isolation primitives are available to driver software. Whether a verified driver (Section 3.3) can enforce strong tenant isolation depends on what the hardware exposes — and right now, we are relying on vendor documentation rather than verified specs for that interface.

2.5.2 Physical Security

Confidential computing has reached the GPU. NVIDIA's H100 supports a hardware-rooted TEE that, in conjunction with CPU-side enclaves (Intel `TDX`, AMD `SEV-SNP`), can encrypt model weights in transit and at rest on the device, with attestation that the firmware and driver stack have not

been tampered with. This matters for AI because model weights are the most valuable artifact in the ecosystem — worth billions in training compute — and multi-tenant cloud deployments expose them to the cloud operator's entire software stack. But confidential computing protects against software adversaries with root access; it does not protect against physical adversaries with access to the hardware.

Modern AI accelerators use High Bandwidth Memory (HBM) stacked on a silicon interposer — the GPU die and HBM stacks sit side by side on a shared substrate connected by thousands of micrometer-scale copper traces. This interposer is a physical interface carrying model weights and activations at terabytes per second, unencrypted in current designs. An attacker with physical access to the package could, in principle, probe these traces or tap the interposer to exfiltrate data. This is not a low-effort attack — it requires lab equipment and destructive or semi-destructive access to the package — but for a state-level adversary targeting a frontier model worth billions, it is not out of scope. HBM encryption at the interface level is not yet standard; until it is, the physical package boundary is the confidentiality boundary.

Side-channel attacks add another dimension. [54] showed that the magnetic flux from a GPU's power cable — measurable with a $3 induction sensor — betrays the detailed topology and hyperparameters of a neural network being evaluated, achieving 96.8% classification accuracy for identifying network layers on an Nvidia Titan V. [55] demonstrated that electromagnetic emanations from GPU DVFS (dynamic voltage and frequency scaling) activity penetrate walls and can be captured at 3+ meters, enabling website fingerprinting and keystroke timing attacks. Scale these techniques to a data center with thousands of GPUs running a single training job, and the EM signature is correspondingly richer. TEMPEST-class shielding is well-understood for traditional compute environments, but GPU training clusters dissipate megawatts of power with correspondingly strong emanations, and it is not clear that standard data center construction provides adequate EM containment. This is an area where the AI security community has barely begun to assess the threat surface.

Counterfeit and recycled chips are a supply chain problem that predates AI but gains new significance when the chips in question carry model weights. The Semiconductor Industry Association estimates counterfeit semiconductors cost the U.S. industry roughly $7.5 billion a year in lost revenue [56]; the most common attack is remarking — taking a chip desoldered from e-waste, sanding the package markings, and relabeling it as a higher-grade or newer part. For commodity components this causes reliability failures; for security-critical components in a training cluster, it could mean a node with degraded or compromised silicon participating in a distributed training run. Detection methods exist — on-chip aging sensors ("odometers"), power side-channel profiling, X-ray inspection of die markings — but none are deployed systematically for GPU procurement at scale. The assumption is that buying from authorized distributors eliminates the problem, which is true until it isn't.

The geographic concentration of semiconductor manufacturing is a systemic risk that no amount of formal verification can address, but it shapes the threat model for everything else in this section. TSMC fabricates over 90% of the world's most advanced chips [57], including every NVIDIA AI accelerator. All of this capacity sits in Taiwan. A single earthquake, a fabrication contamination event, or a military conflict in the Taiwan Strait would halt frontier AI hardware production globally. The CHIPS Act and TSMC's fabs in Arizona, Japan, and Germany are slowly

redistributing this concentration, but leading-edge capacity outside Taiwan remains years away from volume production. Advanced chip packaging — the 2.5D interposer and `CoWoS` technology that makes HBM-equipped AI accelerators possible — is even more concentrated than fabrication itself. For security planning, this means the physical supply chain has a single point of failure that cannot be engineered around with software or formal methods; it can only be mitigated through industrial policy and supply chain diversification.

For training clusters of 10,000+ GPUs, the physical supply chain itself becomes a target at the system level, not just the component level. Every node in the cluster must be attested before it receives model weights or gradient updates. Every network link between nodes carries data that, if intercepted or modified, could poison a training run or exfiltrate a model. Physical tamper-evidence for installed hardware — seals on server chassis, tamper-detecting rack enclosures, logged physical access — is standard practice in classified environments but not in commercial data centers running AI training workloads. As frontier model training costs reach hundreds of millions of dollars, the gap between the physical security posture of a commercial hyperscaler and the value of the assets it protects is widening. FPGA-based network taps (Section 3.8) address the link-level problem: a tap with a formally verified packet-processing pipeline can guarantee that it faithfully mirrors traffic without injection or modification — a property that no software tap running on a general-purpose OS can credibly claim.

3 Tractable Problems

The problems below split into two kinds. **Enablers** are problems whose resolution unbottlenecks a class of downstream work. Adversarial robustness of formal methods (Section 3.1) is the clearest example: if an ITP's soundness can be broken by a sufficiently clever agent, then every verified artifact produced by that ITP inherits the vulnerability. Governance of neuralese proof stacks is similar — if proof oracles start emitting proofs in representations humans cannot audit, spec elicitation and validation become unsolved problems for everything else in this document. Solving an enabler does not itself produce a deployable artifact, but failing to solve it degrades the value of the widgets that depend on it. **Widgets** are concrete, scoped deliverables. Each widget stands on its own as a security improvement to a specific layer of the stack. Widgets are where the postdoc-years get spent and where the artifacts ship. The enabler/widget distinction matters for prioritization: an enabler with many dependents should be addressed early, and a widget whose value is contingent on an unsolved enabler should be sequenced accordingly.

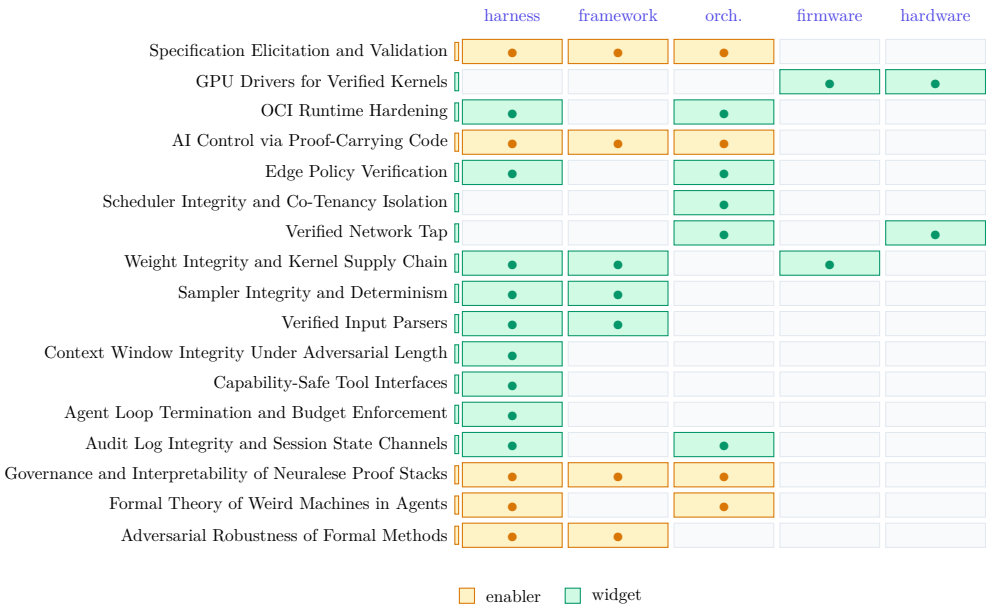

Figure 8: Tractable problems versus the stack layers they touch. Amber rows are enablers; green rows are widgets. A problem can touch more than one layer; most do. Section ordering matches the file include order in this chapter.

3.1 Adversarial Robustness of Formal Methods

Stack layers: Software & ML Framework | Execution Harness

Blocks: Malicious Model

`Agda`'s issue label "false" on GitHub sits at the time of this commit at 10 open and 76 closed. `Agda`'s issue label "false" tracks the *proofs of false* that Agda allows or has allowed. `Rocq` runs the same play under a different name: the `critical` label is reserved for proofs of false. One asks, "isn't the whole point of a type theory that it be sound?"

So you see we have a problem. If ITP and other FM tools are not adversarially robust, scheming or reward hacking AIs will readily leverage novel zerodays to violate security properties.

[58] discusses some of this to set up the `Lean` kernel arena. TODO: elaborate.

Broadly, soundness issues in proof assistants arise from the tension between expressivity and ease of use on one side and consistency on the other. Every concession to ergonomics — definitional extensions, universe polymorphism, coinduction, native compilation of the reduction machine, opaque conversion shortcuts — is a place where the kernel can drift away from the logic it was supposed to implement.

TODO: zero in on `Lean` specifically, (James Henson?), discuss governance challenges and the slippery definition of the problem.

3.1.1 Solution/project Sketch

Soundness bugs often arise due to the extreme complexity of a dependent type theory, who's proof checker we call a kernel (5k LoC in Lean, 10-30k LoC in Rocq). One idea would be to express the language of the ITP, even the dependently typed one, in a simpler theory with a smaller kernel.

So you could, in principle, simply write a model of your complicated dependent ITP in a simple, well-trusted target and check proofs there.

What you want is a system in the LCF tradition (Milner's Logic of Computable Functions), whose defining discipline is that theorems are an abstract datatype and the only way to manufacture one is through a small fixed set of inference rules. That discipline is what makes the trusted core small, and it's preserved across systems with otherwise quite different logics: `HOL Light` [59] and `HOL Zero` [60] (classical higher-order logic), `Candle` [61] (HOL Light verified down through `CakeML`), and `Milawa` (a bootstrapped first-order prover in the `ACL2` lineage [62]). `HOL Zero` gets the trusted core down to a few hundred lines. Push further — verify the logic down to a large cardinal axiom, plus a model of the hardware, plus a small wrapper for actually running things — and rough estimates put the state of the art around 10k lines of spec.

One approach, then, is to pick something simple and well-trusted — ZFC or a variant, or a minimal LCF-style higher order logic, augmented with a large cardinal / large universe axiom — and reduce a working ITP all the way down to that. Useful prior art and reference points: `lean4lean` (a verifier for `Lean 4` written in `Lean 4`) and the `MetaRocq` (formalised metatheory of `Rocq` inside `Rocq`).

3.2 Specification Elicitation and Validation

Stack layers: Execution Harness | Software & ML Framework | Orchestration & Cloud

Blocks: Malicious Model

If proofs are cheap and specs are expensive, then specs are the bottleneck. Every widget in this document terminates in a spec: a verified sampler is verified against *some* statement of what a sampler should do, an OCI runtime is hardened against *some* threat model, an IAM policy is checked against *some* notion of least privilege. The spec is where human intent meets machine-checkable formalism, and it is the part that no proof oracle can write for you — or rather, if a proof oracle writes it for you, you have quietly handed over the thing you were trying to keep.

Two subproblems:

- **Elicitation.** Getting the spec out of the stakeholder's head and into a formal language, without losing the parts they could not articulate. The classical software-engineering requirements-gathering literature is relevant but insufficient — we need elicitation techniques that produce artifacts a kernel can consume, not prose a PM can sign off on.
- **Validation.** Convincing yourself that the spec you wrote is the spec you meant. A proof of `P` against the wrong `P` is worse than useless: it launders a bug into a theorem. The property-based testing tradition [63] and the spec-mining tradition [64] are the two complementary handles we have on this — the first forces you to execute the spec against concrete inputs, the second infers candidate specs from observed behavior for the human to ratify or reject.

TODO: discuss the relationship to Section 3.16 — if the proof substrate is itself opaque, spec elicitation becomes unsolvable rather than merely hard, which is why we treat that as a separate enabler.

TODO: discuss differential specification (two independently elicited specs, checked for agreement) as a validation technique.

3.2.1 Solution/project Sketch

TODO: flesh out. Candidate directions: interactive spec-synthesis from examples, lightweight formal methods for spec sanity-checking, spec-level mutation testing, and human-factors work on what kinds of formal notation domain experts can actually read and write.

3.3 GPU Drivers for Verified Kernels

Stack layers: Firmware & Low-Level Systems | Hardware & Physical Supply Chain

Blocks: Co-tenant | Rogue Insider

Multi-tenant GPU workloads today run on hypervisors whose TCB is orders of magnitude too large to verify, and whose GPU-facing driver is proprietary kernel code running at ring 0 (Section 2.4.2). Two verified microkernels are candidates for hosting an ML-grade GPU stack: `seL4` [65], whose functional-correctness proof is the most complete but which has no GPU driver support at all, and `NOVA` [4], whose partial proof covers concurrency and weak memory and whose microhypervisor architecture is explicitly designed for the host/guest split a GPU-passthrough workload needs.

The research methodology for verifying a driver — separation-logic proofs against an abstract hardware model — is settled [40], [41], [42]. The real blocker is that the abstract hardware model does not exist. No GPU vendor publishes a machine-checkable specification of their command processor, MMU, or DMA engine; Peter Sewell's REMS Group has the most accurate public modeling work but is scoped to the CPU ISA [43]. The tractable problem is accordingly two-sided: produce a formal model of a GPU command-submission interface at a fidelity that supports proof, and prove a driver against it.

3.3.1 Solution/project Sketch

Start with the smallest useful surface: the command-submission path of a single open-source GPU stack (`NVK`/`Nouveau` on NVIDIA, or an `AMDGPU` subset). Specify the command-ring state machine in `Rocq` or `Lean` at a level of detail that admits noninterference claims across tenant contexts — ring-buffer consistency, IOMMU mapping integrity, context-switch scrubbing. Prove a reference driver (runnable under either `seL4`'s `sDDF` or `NOVA`'s userspace driver model) against that spec, with the security property being *no sequence of guest-supplied command packets causes the driver to program an IOMMU mapping or issue a DMA outside the guest's declared memory region.* Two natural stopping points: a verified command-submission module with a stubbed-in hardware model, which is shippable on its own as a reference; and the same driver proved against a model co-developed with the vendor or with REMS, which is the research contribution.

3.4 OCI Runtime Hardening

Stack layers: Orchestration & Cloud | Execution Harness

Blocks: Malicious Model | Co-tenant

The standard container runtime, `runc`, implements the OCI specification atop Linux namespaces and cgroups. It was never designed to resist an adversary *inside* the container, and the CVE record reflects this. When the workload in question is an AI agent with tool use (i.e. one that can write and execute arbitrary code), the gap between "process isolation" and "security boundary" becomes the whole game.

Recent benchmark work has started to quantify the gap. [66] measures frontier LLM capabilities for container sandbox escape directly, treating the model as a red-team operator. The UK AISI sandboxing toolkit [67] and its Docker Sandbox Breakout CTF [68] provide standardized escape scenarios built on `Inspect AI` [69]. [70] introduces BashArena, a control setting specifically for highly privileged agents. BoxArena [71] inverts the usual offensive framing: instead of asking "can this agent escape?", it fixes the attacker model and varies the OCI runtime, producing a *defensive* leaderboard. Five attack surfaces (filesystem, socket, process, network, syscall) are tested across runtimes like `runc`, `gVisor`'s `runsc`, `crun`, and `Kata Containers`.

The pattern across all of these is convergent: empirical red-teaming finds holes, patches get applied, and next quarter a new escape surfaces. A sufficiently capable agent should be assumed to find any flaw in its containment. This is where formal methods enters.

3.4.1 Solution/project Sketch

What would a *formally verified OCI runtime* look like? The lineage runs through the `CLInc` stack [72] (end-to-end verified from gate-level hardware through compiler to application) and `seL4` [65] (functional correctness of a microkernel, machine-checked in `Isabelle/HOL`). A verified OCI runtime would prove that the namespace and capability isolation enforced by the runtime matches a formal policy – that no sequence of syscalls from inside the container can violate the confinement invariant.

This is a large artifact, but it decomposes into tractable pieces: a formal model of the relevant Linux ABI surface (the 80 syscalls a hardened container actually needs), a specification of the confinement policy (drop all capabilities, read-only rootfs, `seccomp` allowlist, no-new-privileges), and a proof that the runtime's state machine preserves the policy across all reachable states.

The AI control literature [73], [74] adds another dimension: even with a verified runtime, you want a *monitor* – a second model or policy that reviews agent actions before they reach the kernel. The combination of a verified isolation boundary and a scalable monitor protocol is stronger than either alone, and the monitor's job gets easier when the boundary it sits behind has a proof.

3.5 AI Control via Proof-Carrying Code

Stack layers: Execution Harness | Software & ML Framework | Orchestration & Cloud

Blocks: Malicious Model

Proof-carrying code [75] and AI Control [73], [74] are two literatures about the same problem: keep an untrusted producer on the far side of an interface and let only a small trusted checker decide what gets through. PCC's answer is that every proposed action arrives tagged with a machine-checkable certificate; the trusted base is the proof kernel and nothing else. AI Control's answer is that a weaker trusted monitor, backed by a scarce audit budget, decides per-action on a safety–usefulness Pareto frontier. The deterministic midpoint — the untrusted agent emits code plus a proof in a standard ITP, the boundary rejects anything that does not typecheck, and the trusted base collapses to the kernel plus a human-written specification — has been sketched at prototype scale [76] but not stress-tested.

The question this enabler addresses is whether that synthesis can actually pay for itself. Classical PCC was designed for a world where proofs were written by compiler-level procedures on statically-known properties. Extending it to a world where the code producer is a frontier model and the safety property is "this action does not compromise the serving stack" raises a quantitative issue:

the *verification burden* k — the ratio of proof-synthesis cost to program-synthesis cost. A world where k is small is a world where AI-control-via-PCC is the right architecture for most widgets in this document. A world where k is large — or where it is small only for spec-shapes nobody writes in practice — is a world where those widgets have to be verified individually, outside any unified control protocol.

Three open subproblems determine where k lands.

Proofs as a matter of degree. Current type checkers give a binary signal: the proof closes or it does not. A partial proof that is ninety percent of the way to closure is indistinguishable, to the kernel, from a partial proof that is ten percent there. A control protocol that could read *how close* a submitted proof is to closing — via a learned value function over tactic sequences, a conditional-probability estimate on the remaining subgoal structure, or a simpler admissibility heuristic — could admit "warm" proofs under bounded risk and reject "cold" ones as likely unprovable. This replaces the all-or-nothing gatekeeper with a graded one, and the grading itself becomes a research target.

Specialist blue-team models. Nothing in the AI Control framing requires the trusted checker to be a general model. A small model fine-tuned on a specific Hoare fragment, with custom tactics targeting a specific surface (context-window bookkeeping, permission-attenuation lemmas, IFC label discharge), exploits the asymmetry that verification is sometimes provably cheaper than generation. The training cost is upfront and amortized over deployment; the deployment cost is low by construction. The open question is how much of the security-relevant spec space admits specialists this way.

Upfront versus ongoing investment. The seL4 project [65] spent roughly 2.2 person-years on kernel design and coding, 9.0 on reusable infrastructure and tooling, and 11.0 on kernel-specific proof engineering — roughly a $1:1$ ratio of better-swords to more-fencing. In a PCC-for-AI world, who pays for the tooling half? If the red team pays — compliance as a cost of doing business, on the SOC-2 model — the original Ctrl constraint that *blueteam_budget* much-less-than *redteam_budget* dissolves, and the verification-burden arithmetic changes sign. If the blue team pays, the constraint binds and the first two subproblems become load-bearing.

3.5.1 Solution/project Sketch

Four threads, sequenced by how much they unblock.

Verification-burden measurement at scale. Measure k on the spec shapes that actually matter for this document: context-window allocation (Section 3.12), sampler conformance (Section 3.10), tool-composition safety (Section 3.13, Section 3.17), and weight-load refinement (Section 3.9). Produce k distributions per spec class, per model, per year. This is an ongoing benchmark, not a one-shot paper.

Warmness metrics and graded gatekeeping. Build partial-proof signals into a control protocol. Start with `Lean` tactic-sequence value functions trained on human and synthetic proof traces, extend to subgoal-structure embeddings, and evaluate as a replacement for binary pass/fail in a backdoored-code testbed in the style of [73]. The deliverable is a graded gatekeeper with a published ROC curve on audit-budget-vs-catastrophe-rate.

Specialist-checker training. Pick one narrow spec surface — say, Hoare logic over the context-assembly allocator — and train a small model specifically for its proof obligations with custom tactics. Measure the specialist's k against a general model's k. Repeat across surfaces until the specialist-viability boundary is clear.

Control protocols as a programming language. Speculative and out-of-scope for the near-term, but worth naming. The Dolev-Yao model [77] reduced cryptographic-protocol reasoning to logic programming and absorbed a class of attacks into definitional unreachability. An AI-control analogue — a process calculus in which resampling, deferral, and trusted-editing primitives have first-class semantics and attacker capabilities are syntactic — would let protocol designers discharge parts of the soundness argument at design time rather than per deployment. PLT or FV researchers looking for an entry point into AI safety should consider this direction.

3.6 Edge Policy Verification

Stack layers: Execution Harness Orchestration & Cloud

Blocks: External User Malicious Model

The API gateway in front of an inference service handles authentication, rate limiting, and authorization before a request ever reaches the model. Each of these is a distinct protocol with distinct failure modes, and they interact. JWT algorithm confusion [78] let attackers forge tokens by switching from RSA to HMAC verification — a bug that lived in production libraries for years because the state space of the auth handshake was never systematically explored. OAuth 2.0 flows compound the problem: the redirect-based token exchange has enough moving parts (authorization codes, refresh tokens, PKCE challenges, session binding) that implementation errors routinely produce exploitable states. These are not novel observations, but the standard response — unit tests and penetration testing — samples the state space rather than covering it.

Rate limiting in distributed deployments is a second, largely independent class of bug. The typical architecture uses distributed counters (`Redis`, or an in-process sliding window with gossip) to enforce per-key quotas. Under concurrent writes, race conditions allow burst traffic to exceed policy. The failure is often silent: the system reports correct enforcement while the actual request count drifts above the limit. Temporal logic can express the property you actually want — "over any window of length T, the count for key k does not exceed N" — and model checking can verify that a given counter protocol satisfies it under all interleavings, not just the ones your load test happened to hit.

The third piece is authorization for agent-originated requests. In agentic deployments, the model generates tool calls — HTTP requests, database queries, shell commands — that carry the *human* session's credentials. The model acts on behalf of the user but is not the user; it is a textbook confused deputy. If the routing logic does not attenuate authority, a prompt injection or jailbreak that causes the model to emit an unexpected tool call inherits whatever permissions the session holds. The question is whether the set of actions reachable through model-generated requests is a strict subset of the actions the human intended to authorize, and this is a containment property amenable to formal analysis.

These three concerns — authentication correctness, quota enforcement, and agent authority attenuation — share a common trait: each is a small protocol with a large state space, running at a

trust boundary. They are individually tractable for current FM tools but rarely analyzed together, even though a bug in one can undermine the guarantees of the others.

3.6.1 Solution/project Sketch

The project decomposes into three workstreams that share a common formal model of the gateway's request lifecycle. First, model the authentication state machine (`JWT` issuance, validation, algorithm selection, key lookup) in `TLA+` and verify that no reachable state admits a token accepted under an unintended algorithm or expired credential. Extend this with a `ProVerif` [79] or `Tamarin` [80] model of the full OAuth 2.0 flow as instantiated by the gateway, producing machine-checked proofs of token secrecy and session binding. Second, express the rate-limit policy as a temporal logic formula and verify the distributed counter implementation against it in `TLA+`, covering all process interleavings and crash-recovery scenarios. Where the counter uses shared mutable state, apply `Iris` or another concurrent separation logic to prove the monotonicity invariant (the counter never decreases except at window expiry) under concurrent writes. Third, model agent authority as a capability system in `Alloy`: the human session holds a capability set, the model's tool-call interface holds an attenuated subset, and the routing logic mediates between them. Verify that no sequence of model-generated requests can reach a capability outside the attenuated set — a direct confused-deputy analysis of the gateway's forwarding rules. The final deliverable is a verified reference gateway configuration (or shim) with proofs covering all three properties, plus a test harness that regression-checks deployed gateways against the formal spec.

3.7 Scheduler Integrity and Co-Tenancy Isolation

Stack layers: Orchestration & Cloud

Blocks: Co-tenant Rogue Insider

The dominant real-world mitigation for co-tenant attacks is not a scheduler policy but a purchase order: hyperscaler customers with serious confidentiality requirements buy reserved or bare-metal capacity and opt out of shared tenancy entirely. This section is scoped to the deployment model where that option is not on the table — mid-tier cloud providers, academic and national-lab clusters, and internal multi-team platforms at organizations without per-team hardware budgets. In that setting, the cluster scheduler (`Kubernetes`, `Slurm`, or a custom allocator) decides which physical host runs which job, and the correctness of that decision is load-bearing for any isolation claim the operator wants to make.

The framing matters because the question "which tenant pairs are adversaries" is not answerable a priori. An attacker knows their target; the operator, ahead of time, does not. So the tractable formal problem is not "identify dangerous co-locations" but "given an isolation policy the operator has specified, prove the scheduler enforces it". The placement logic is a constraint solver operating over dozens of dimensions (resource requests, affinity, taints, topology spread, GPU fabric topology), and the interaction between constraints is hard to reason about by inspection. A misconfigured or under-constrained policy can silently permit co-location the operator assumed was forbidden — and the failure is detectable only post-factum, by which point the attacker has already been a co-tenant.

Resource accounting has none of this scoping difficulty. Attribution fraud (one tenant's GPU-hours billed to another) and quota evasion (a malicious job consuming more than its allocated share without detection) apply to every shared-tenancy deployment regardless of whether anyone

is trying to side-channel anyone else. The metering pipeline — usage counters, aggregation, attribution across concurrent job start/stop and preemption — is a distributed accounting protocol with the same class of race conditions that make distributed rate limiting hard, and it admits the same class of formal treatment.

The residual confidentiality story — cache side channels (Flush+Reload [81], Prime+Probe [82]) against a co-resident victim — then becomes a secondary application of the same placement model: given a formally enforced policy, one invariant of that policy can be "no two jobs labeled *mutually-distrusting* share an LLC domain or GPU memory fabric", and the same verification discharges it.

3.7.1 Solution/project Sketch

Two workstreams, both grounded in policy-enforcement rather than threat identification.

First, accounting correctness. Model the metering pipeline (counters, aggregation, attribution) as a distributed state machine and prove with concurrent separation logic (`Iris` or equivalent) that the accounting invariant holds: every GPU-second is attributed to exactly one tenant, and the sum of attributed usage equals actual usage, under concurrent job start/stop, preemption, and node failure. This is universally applicable and does not depend on any adversary-identification assumption. The deliverable is a verified reference metering library with a concurrency-safe aggregation protocol, usable as a drop-in for existing schedulers.

Second, placement-policy enforcement. Model the cluster topology (hosts, LLC domains, GPU memory fabrics, network segments) and the operator-authored isolation policy (expressed as relational constraints over tenant labels and topology resources) in `Alloy` [83]. The verification question is not "is this policy the right policy" — that is a business decision the operator owns — but "does the scheduler's constraint solver, across all feasible cluster states and job arrival sequences, only produce placements that satisfy the stated policy". Counterexample traces from bounded model checking become a conformance test suite against a concrete scheduler (e.g., `Kubernetes` with a given set of affinity and taint rules). The deliverable is a policy-specification language, an `Alloy` model of a representative scheduler, and an audit shim that checks live placement decisions against the model.

3.8 Verified Network Tap

Stack layers: Orchestration & Cloud | Hardware & Physical Supply Chain

Blocks: Network MITM | Physical Adversary

The control plane that decides what flows where on a GPU cluster fabric — SDN controllers, fabric managers, tenant routing tables (Section 2.3.2) — is the same piece of infrastructure whose compromise is the threat. An operator's own logs about what crossed which link cannot be evidence in a threat model where the operator's logging stack is on the attacker's side of the trust boundary. Independent observation on the fiber itself is the difference between "the controller says traffic between tenants A and B was partitioned" and "no such packet appeared on the inter-pod link during the audit window."

[84] develop this as the verification primitive for AI-datacenter compliance under mutual distrust between operator and external verifier. Their cost figures are the encouraging part of the picture: a north-south tap at the datacenter edge captures all external traffic at well under 0.01% of facility upfront cost; an east-west storage-fabric tap with full hash-and-timestamp capture sits at

0.3-0.75%; compute-fabric sampling via optical circuit switches at 0.2-1.5%. FPGA capture at 400 Gbps and above already has precedent in financial-regulation hardware (Arista 7130, Deutsche Börse). [85] adds the optical reality the spec has to live with: passive splitters add no latency but eat link budget — a 1 dB loss can break a 53 Gbaud link — while active OEO taps regenerate the signal at roughly 20 ns latency and tens of watts but introduce a new failure point on the live link. The same artifact serves an internal threat model (compromised SDN controller, dishonest fabric manager) and an external one (treaty verifier, regulatory audit, third-party red team); the LTL contract below is identical, and the threat-model split surfaces only in key custody and in who is allowed to read the monitor leg — policy rather than mechanism (Section 1.4.1).

3.8.1 Solution/project Sketch

The deliverable is the LTL contract below plus an open-source RTL reference design proven against it, for an active optical-electrical-optical tap at 400 Gbps with a path to 800 Gbps. FPGA is the expected deployment target: [84]'s 400 Gbps capture precedent (Arista 7130, Deutsche Börse) is FPGA-based, and the audit story benefits from a re-flashable bitstream tied to a published proof. But the contract is platform-agnostic — an ASIC implementation that meets P1-P6 is equally valid, and an open-source RTL reference is what gets verified, not any particular bitstream. Three pieces: a hardware specification in `Cryptol` or `SystemVerilog` with proofs in `SymbiYosys` or `Kami` against the contract below; a tamper-evident reference design with externally inspectable interfaces; and an out-of-band verification cluster that consumes the monitor stream and runs compliance checks asynchronously, so the wire-side hardware stays simple enough to verify while sophisticated analysis happens elsewhere.

Let `live_in(p, t)` mean packet `p` was clocked into the tap on the live ingress at time `t`; `live_out(p, t)` that it was clocked out on the live egress; `mon(r, t)` that the monitor leg emitted a record `r` at time `t`; and `fail(t)` that the tap has self-reported a fault. Let `D` and `D'` be the forwarding and mirroring deadlines in clock cycles, and let `image(p)` be the deterministic projection of `p` into the monitor record format (header parse + timestamp + payload hash, per [84]).

Using $\Box$ for "always", $\Diamond$ for "eventually", $\blacklozenge$ for the past-time "once", and $\Diamond_{\le D}$ for "eventually within D steps":

$$\Box(\text{live\_in}(p) \rightarrow \Diamond_{\le D}\ \text{live\_out}(p)) \qquad \text{(P1) forwarding completeness}$$

$$\Box(\text{live\_out}(p) \rightarrow \blacklozenge\ \text{live\_in}(p)) \qquad \text{(P2) forwarding fidelity}$$

$$\Box(\text{live\_in}(p) \rightarrow \Diamond_{\le D'}\ \text{mon}(\text{image}(p))) \qquad \text{(P3) mirror completeness}$$

$$\Box(\text{mon}(r) \rightarrow \blacklozenge(\exists p.\ \text{live\_in}(p) \wedge \text{image}(p) = r)) \qquad \text{(P4) mirror soundness}$$

$$\Box(\text{live\_out}(p) \rightarrow \neg\ \text{depends-on}(p, \text{mon-input})) \qquad \text{(P5) no back-channel}$$

$$\Box(\text{fail} \rightarrow \Box(\text{live\_in}(p) \rightarrow \Diamond_{\le D}\ \text{live\_out}(p))) \qquad \text{(P6) fail-to-wire}$$

P1 and P2 together discharge the transparency obligation against the live forwarding path: nothing dropped, nothing synthesized. P3 and P4 discharge the monitor-leg faithfulness obligation: no missing packets, no fabricated ones. P5 is a hyperproperty rather than a pure LTL formula — a noninterference statement that the live-out signal does not depend on any input received via the monitor leg — and is the property that defeats the verifier-collusion attack in [84]'s threat model. P6 is the fail-open invariant from [85]: a tap component failure must downgrade the device

to a transparent pass-through, not a link-cut, because the live link is load-bearing for production traffic.

The projection `image(p)` is where the spec interacts with policy. For east-west compute-fabric taps, `image(p)` is a header parse plus a SHA-3 payload hash — the verifier learns timing and routing without payload contents. For north-south edge taps, the operator may retain session keys for post-hoc decryption under audit. Both factor through the same LTL contract; only the definition of `image` changes, which is the spec-implementation separability principle from Section 1.4.3.

Two natural stopping points. A verified single-port tap with a stub fail-detection path, proven against P1-P6 modulo a hand-modeled SerDes layer, is shippable as a reference design. The same tap proven against a co-developed model of a real 400G PCS/SerDes is the research contribution; this is the same hardware-model gap that blocks GPU driver verification (Section 2.4.2). Multimode fiber is out of scope — the link budget does not admit passive splitting at high baud rates [85] — and the industry trend toward single-mode-only AI fabrics is the precondition this widget ships against.

3.9 Weight Integrity and Kernel Supply Chain

Stack layers: Execution Harness | Software & ML Framework | Firmware & Low-Level Systems

Blocks: Supply Chain | Rogue Insider

Model weights are the highest-value static artifact in an ML deployment. A frontier model's checkpoint runs tens to hundreds of gigabytes, sharded across a distributed filesystem, and any bit-flip or deliberate modification in those shards changes the model's behavior in ways that are hard to detect at inference time. The threat model is straightforward: an adversary with write access to the storage layer — compromised NFS node, supply-chain attack on a model registry like `Hugging Face`, or a privilege escalation on the serving host — rewrites a weight shard. Existing defenses amount to hash-checking at load time. But the loader itself is a large, unverified C++/Python codebase, and the check is only as trustworthy as the code that performs it. A compromised loader can skip the check, and a hash collision (unlikely but not formally excluded) defeats it silently.

A second, orthogonal problem lives one layer deeper. The compute kernels that execute the forward pass — matrix multiplications, attention, activation functions — are supplied by hardware vendors as precompiled binaries. Nvidia's `cuBLAS` and `cuDNN` are closed source. AMD's `MIOpen` is partially open. In both cases, the optimized kernel that actually runs on the GPU is not the code anyone audited. A compromised kernel could introduce targeted numerical perturbations (a backdoor that activates on specific input patterns), exfiltrate data through covert timing channels, or silently degrade accuracy. You cannot inspect what you cannot read, but you *can* check what it does against what it should do.

These two attack surfaces — weight tampering and kernel compromise — compose badly. If both the weights and the kernel are untrusted, the search space for an auditor explodes. Pinning one of them down with a formal guarantee makes the other tractable.

3.9.1 Solution/project Sketch

The weight-loading pipeline should be verified end-to-end: from the filesystem `read()` call, through hash computation and comparison, to the final `cudaMemcpy` (or equivalent) that places weights in device memory. The postcondition is that the bytes in GPU memory are identical to the bytes

committed by a known-good hash. This is a refinement proof — show that the runtime representation is a faithful image of the storage representation, and that no intermediate step can silently substitute content. The verified loader then extends the remote attestation chain: the attestation report already covers firmware and boot integrity, so adding a weight-hash measurement means the serving infrastructure can refuse to run inference unless the full chain checks out. Model-checking the serving state machine (load, attest, serve) confirms that no path reaches the first forward pass without a valid attestation.

For the kernel supply chain, the tool is translation validation [86]. Each kernel gets a behavioral contract: "given input matrices X and W, the output is XW up to floating-point error ε." A validator checks the compiled binary against this contract without needing source code — it works on the artifact, which is exactly what you want when the vendor won't ship source. As a stepping stone, differential testing against a formally verified reference implementation (a simple, correct matmul in, say, `Fiat Cryptography`'s [87] style) catches gross violations cheaply. Translation validation catches subtle ones with a proof.

3.10 Sampler Integrity and Determinism

Stack layers: Software & ML Framework Execution Harness

Blocks: Rogue Insider Malicious Model

The sampler is the last piece of code between the model's output logits and the token that reaches the user. It applies temperature scaling, top-k or top-p filtering, and then draws from the resulting distribution. This is a small amount of code — a few hundred lines in a typical inference server — but it sits at a chokepoint. An adversary who controls the temperature parameter, the random seed, or the filtering threshold can bias the output distribution without touching weights or the forward pass. The attack surface is not hypothetical: inference servers expose these parameters via API, configuration files, and environment variables, and the sampler's internal state (particularly the PRNG) is rarely isolated from the rest of the serving process.

The scoping here is what makes the problem attractive. Unlike weight loading or kernel validation, the sampler is small enough to verify. The specification is a probability distribution parameterized by (logits, temperature, top-k, top-p, seed), and the property to check is that the implementation samples from exactly that distribution — no hidden state, no side-channel inputs, no drift from the spec across calls.

3.10.1 Solution/project Sketch

Specify the sampler as a probabilistic program and verify it with a probabilistic model checker (`PRISM` [88] or `Storm` [89]). The key property: for any input logit vector, the output token distribution is within a specified KL-divergence bound of the theoretical distribution defined by the declared parameters. This catches both outright bugs (off-by-one in top-k filtering) and subtle manipulation (PRNG state leaking information across requests). Separately, prove data-flow integrity on the implementation — that the sampler is a pure function of its declared inputs with no dependence on mutable global state or memory reachable from other threads. This is a standard information-flow analysis, tractable for code this size. The combination of distributional correctness and data-flow purity gives you a sampler you can trust at the boundary between model and user.

3.11 Verified Input Parsers

Stack layers: Execution Harness | Software & ML Framework

Blocks: External User | Malicious Model

Every boundary in the AI serving stack where text is parsed or transformed is a security boundary. The abuse classifier preprocessor tokenizes, truncates, and encodes user input before classification — if that pipeline is wrong in any of several subtle ways (non-idempotent truncation, encoding expansion, off-by-one in token windowing), an adversary can craft inputs that the classifier never actually sees. The prompt assembly layer concatenates system instructions, developer instructions, retrieved context, and user messages into a single model input, relying on implicit priority ordering that the model is supposed to respect but cannot enforce. The tokenizer itself — typically byte-pair encoding — is deterministic but has never been formally specified; BPE admits multiple encodings for the same string and includes control-character tokens that downstream components may interpret specially. And on the output side, tool-call parsers extract structured invocations from free-form model text, where any confusion between "model is describing a tool call" and "model is issuing a tool call" is a direct security breach.

These are not four separate problems. They are one problem with four instances: *unverified parsers at trust boundaries*. The LangSec movement [90] has argued for years that ad-hoc input handling is the root cause of most exploitation. In the AI serving stack the argument applies with unusual force, because the inputs are adversarially chosen (user prompts), the transformation pipelines are subtle (BPE, priority-based concatenation), and the consequences of parser confusion range from safety-filter bypass to arbitrary tool execution.

The formal methods tooling for this class of problem is mature. `EverParse` [91] produces verified zero-copy parsers from format specifications, with extraction to C. Parser-combinator libraries in `Lean` and `Coq` can produce proofs of totality, soundness, and completeness alongside executable code. The properties we need — that truncation is idempotent, that encoding is non-expanding, that user-supplied characters cannot occupy system-privileged token positions, that tool-call extraction is sound and complete with respect to a grammar — are all first-order properties over finite structures, well within reach of existing verification technology.

What has been missing is not capability but *targeting*: nobody has written the formal grammars for these specific interfaces and connected them to the actual serving code. The gap is an engineering gap, not a research gap.

3.11.1 Solution/project Sketch

Frame this as a family of verification targets sharing a common methodology: for each trust boundary, (1) write a formal grammar of the interface, (2) implement a verified parser (`EverParse` for C-level components, `Lean` parser combinators for higher-level ones), and (3) prove the relevant security properties.

Abuse classifier interface. Specify the grammar of valid requests. Verify that the preprocessing pipeline is a total function from the grammar to classifier inputs. Prove truncation is idempotent and encoding is non-expanding — i.e. that `preprocess(preprocess(x)) = preprocess(x)` and `|encode(x)| <= |x|` for the relevant notion of length.

Prompt assembly. Define a structured document format with explicit priority zones. Prove non-interference: no user-supplied byte sequence can place tokens into the system-prompt zone. This is a separation property on the assembly function, checkable by symbolic execution or refinement proof.

Tokenizer. Formalize BPE in `Lean` or `Coq`. Prove totality (every byte string has exactly one encoding), surjectivity onto the vocabulary, and that detokenization is a left-inverse of tokenization. Define a "safe" token subset and verify that user-controlled input maps only into this subset.

Tool-call parser. Define the tool-call grammar as a formal language. Implement as a verified parser. Prove soundness (anything extracted is a valid tool call per the grammar) and completeness (no valid tool call in the output is missed). The parser should reject ambiguous outputs rather than guess.

Each of these is a standalone artifact producing a verified `.c` or `.lean` module that can be linked into existing serving infrastructure. The shared methodology means the second target is cheaper than the first, and the fourth is routine.

3.12 Context Window Integrity Under Adversarial Length

Stack layers: Execution Harness

Blocks: External User · Malicious Model

A model's context window is a fixed-size resource. The serving harness fills it by concatenating, in priority order, a system prompt, developer instructions, retrieved context (from RAG or tool outputs), and the user message. When the user message or retrieved context is long enough, something has to be truncated — and in most deployed systems, truncation is last-in-wins or proportional, not priority-respecting. An adversary who controls message length (trivially: just send a long message) or who can influence retrieved-context length (less trivially: poison a document store with padding) can force eviction of the system prompt. Once the system prompt is gone, so are the behavioral constraints it encodes.

This is a resource-scheduling problem in disguise. The system prompt and developer instructions are hard reservations; user input and retrieved context are elastic. The invariant is simple to state: *the system prompt occupies a reserved prefix of guaranteed minimum size, and truncation of lower-priority sections never reduces the system-prompt allocation.* This is a monotonicity property — increasing the length of a lower-priority section can only shrink other lower-priority sections, never the reserved prefix. It is exactly the kind of property that tools like `Dafny` and `Frama-C` are built to verify, because it reduces to an invariant over a bounded integer program (token counts, priority levels, allocation sizes).

The reason this has not been done is that context assembly logic is typically embedded in application code — a few dozen lines of Python in a `LangChain` pipeline or a custom harness — and nobody has extracted a specification from it. But the specification is short, the code is short, and the proof obligation is a loop invariant. This is a low-effort, high-value verification target in the serving stack.

3.12.1 Solution/project Sketch

Model context allocation as a resource-bounded scheduler with a fixed budget of N tokens and k priority levels. Each level has a minimum reservation r_i with $\sum r_i \leq N$. Surplus tokens are

distributed to lower-priority levels in priority order. Specify the allocator in `Dafny` or `Frama-C` and prove two properties: (1) *reservation guarantee* — for all inputs, every priority level receives at least r_i tokens, and (2) *monotonicity* — increasing the requested length of level j does not decrease the allocation of any level i with $i < j$ (higher priority).

Extract the verified allocator as a standalone function that existing harnesses can call to compute truncation boundaries before assembling the context. The function takes token counts per section and returns per-section truncation limits. Integration is a one-line change in any harness that currently does its own ad-hoc truncation.

3.13 Capability-Safe Tool Interfaces

Stack layers: Execution Harness

Blocks: Malicious Model

A single tool call in an agentic harness is usually benign. Read a file, run a shell command, make an HTTP request — each is authorized individually by whatever permission system the harness enforces. The problem is composition: read file + shell exec + HTTP request = exfiltrate a secret key. No single call crossed a policy boundary, but the *sequence* did. This is the weird machines problem [92] applied to AI tool use: individually permitted calls compose into unauthorized capability.

The obvious response — enumerate the tool set, encode it as a process algebra, model-check reachability of a "dangerous state" — has two failure modes that make it a poor shovel-ready target. The dangerous-state predicate is itself the hard part: define it too narrowly (one fixed exfiltration path) and the verification is trivial; define it too broadly (any bit leaving the session) and every nontrivial tool set admits reachable violations. The analysis also targets the tool set *after* it has been assembled — by the time the model checker runs, composition has already happened, and the verifier's only lever is to refuse to ship. A "your agent is weird-machine-complete" diagnosis arrives too late to inform the design.

The alternative is to stop asking "is this tool set safe" and start asking "what does a tool interface look like such that safe composition is the default". Object-capability languages — `E`, `Newspeak`, `Monte`, and at the OS level `Capsicum` [93] and `seL4`'s capability system [65] — have decades of theory on this question, and none of it has been specialized to AI tool dispatch. The ambient-authority assumption pervading current tool protocols (a tool named `file_read` taking a path argument implicitly holds read authority over the entire filesystem, attenuated only by whatever the harness decides to do post-hoc) is exactly the assumption those languages refuse.

This widget is the correct-by-construction arm of the two-arm split von Hippel draws [94] between correct-by-construction tool languages and cheap-proofs-per-deployment. It is worth pursuing not because it displaces the per-deployment path, but because writing down what "safe composition" means in an interface forces the vocabulary that the per-deployment path also depends on. See Section 3.17 for the enabler that carries that vocabulary.

3.13.1 Solution/project Sketch

The deliverable is a tool-interface specification language with two properties. Every tool declares its capability footprint as a typed requirement — what authority it needs, expressed as a labelled region of the session's capability set, not an ambient permission. Composition of tools is constrained

by a flow-type system such that any well-typed agent program satisfies a declared non-exfiltration theorem by construction.

Three pieces.

First, a surface language. Extend an existing tool-declaration format (`MCP` schemas, `OpenAI` function-calling JSON) with capability annotations: each parameter carries a flow label, each return value carries a label, and the tool's behavior is a declared transformation between them. This is `Jif`-style [95] information-flow typing retrofitted onto tool APIs, with the additional constraint that authority is held as first-class capabilities (read-this-file, post-to-this-URL) rather than pool permissions (read-any-file).

Second, a type system. Given a set of annotated tools and an agent program (a trace or policy that composes them), the type checker accepts only compositions in which no high-label value flows to a low-label sink. The novelty is in the labels: they are capability references, not secrecy classes, so the non-interference theorem states that an attacker-reachable channel can only carry data whose provenance the capability graph permits.

Third, a reference implementation and soundness proof. Build the type checker in `Rocq` or `Lean`, prove progress-and-preservation against a small-step semantics of the tool-dispatch loop, and ship a compiler from the surface language to a thin runtime wrapper over an existing agent harness. A well-typed program cannot, by construction, compose the file-read + shell-exec + HTTP-post exfiltration chain, because the flow labels on the file contents do not match the labels the HTTP sink accepts — not because the harness noticed and blocked the call, but because the program would not have type-checked.

What the project does *not* do: it does not decide which capabilities to grant an agent for a given user task. That is a policy question, owned by whoever writes the agent's capability manifest. What the project delivers is a tool interface in which the manifest has teeth — stated capabilities bound behavior by construction, not by runtime check.

3.14 Agent Loop Termination and Budget Enforcement

Stack layers: Execution Harness

Blocks: Malicious Model

Agentic loops run until they terminate, and if they don't terminate, they don't stop. Resource budgets (step count, token count, wall time) are the standard defense, but the enforcement is ad hoc. Three recurring bugs. If the termination check runs *after* tool dispatch rather than *before*, an off-by-one in the budget counter lets the agent squeeze in one extra call — which may be the exfiltration call. If the budget is checked in application code that the agent can influence (e.g., by writing session state that the check reads), the bound is not a bound. If the budget counter is shared across concurrent sub-agents without synchronization, two children can each pass a "budget remaining" check and then both execute, overshooting the parent's budget.

The property is a small liveness claim about a small state machine: every execution eventually reaches a terminal state, the counter is monotonically non-decreasing, and the termination check precedes every dispatch. None of this is hard in principle; it is hard because the invariants are not written down anywhere, and the dispatch loop sits in application code alongside a lot of other concerns. This widget is explicitly orthogonal to capability-safe tool interfaces (Section 3.13) — a

tight budget bounds how much weird computation an attacker can build, but budget correctness and capability attenuation are independent properties with independent proof obligations.

3.14.1 Solution/project Sketch

Specify the harness dispatch loop as a small-step transition system with one rule: if the budget counter is strictly less than the bound, advance by one and dispatch; otherwise halt. Verify as an LTL liveness property in `TLA+`, and verify the implementation against the spec in `Dafny` (managed-language harnesses) or `Frama-C` (C harnesses). The contract is a precondition on the dispatch function and a loop invariant on the agent loop; both are small enough that a reviewer can read them in one sitting. For concurrent sub-agents, model the shared budget as an atomic fetch-and-decrement and prove the monotonic non-negativity invariant with `Iris` or an equivalent concurrent separation logic. The deliverable is a verified reference dispatch loop, a fuzzing harness that stresses the budget-counter path against the spec, and a small `Dafny` library other harnesses can import.

3.15 Audit Log Integrity and Session State Channels

Stack layers: Execution Harness Orchestration & Cloud

Blocks: Malicious Model Rogue Insider

The audit log is the primary accountability mechanism for deployed AI systems. Regulators, red teams, and incident responders all depend on the same thing: a complete, tamper-evident record of what the model did. If log entries can be suppressed, falsified, or selectively deleted, the system is unauditable — and any safety claim that rests on "we can review the logs" collapses. Append-only storage and cryptographic log chaining (in the style of Certificate Transparency [96]) provide tamper *evidence*, but the implementations are not formally verified. The gap between "we use Merkle trees" and "no API call sequence available to the serving infrastructure can modify a committed entry" is exactly the gap formal methods fills.

There is a subtler problem on the same boundary. When a model's output or summary is written back to session state for future context windows, a persistent side channel is created. A misaligned model can encode messages in session state that survive request boundaries — instructions to future invocations of itself, breadcrumbs for capability accumulation, or data staged for later exfiltration. This channel is invisible to the output filter, which inspects the response returned to the user, not the write-back to the session store. The threat model is not hypothetical: any system that persists conversation summaries, scratchpads, or tool-call results across requests has this surface.

The two problems are linked by a common structure: both involve a write path that must satisfy integrity constraints, and both fail silently when the constraints are violated. A corrupted log and a poisoned session store both look normal from outside the system until someone audits the contents — and the audit itself depends on the log being intact.

3.15.1 Solution/project Sketch

For log integrity, build an `Alloy` model of the storage API and permission system. The specification: no sequence of API calls available to any principal in the serving infrastructure (model, harness, orchestrator) can modify or delete a committed log entry. This is a finite-state check over the permission graph and the storage state machine; `Alloy`'s bounded model checking handles it directly. Separately, specify logging *completeness* as a `TLA+` liveness property on the serving state

machine: every request that reaches forward-pass execution produces exactly one log entry, and that entry is committed before the response is returned to the caller. The completeness property catches suppression; the `Alloy` model catches tampering.

For session state, apply information flow analysis. Model the session store as shared memory and the write-back function as the interface under test. The noninterference property: the content written to session state must be a function only of authorized inputs (the model's declared output schema, the user's message, the tool results). Any information flow from the model's hidden state to the session store that is not mediated by the declared output path is a violation. This is verifiable as a pre/postcondition contract on the write-back function — specify the allowed content, prove the function produces only that content. `Dafny` is a natural fit: the write-back function is small, the specification is a refinement type on the output, and the verifier handles the quantifier reasoning automatically.

3.16 Governance and Interpretability of Neuralese Proof Stacks

Stack layers: Execution Harness | Software & ML Framework | Orchestration & Cloud

Blocks: Malicious Model

table stakes is that proofs are cheap. We assume a proof oracle will make code and proofs that they're correct too cheap to meter.

One idea is that specs remain expensive, though, because we always want a human in the loop. Elicitation and validation of specs is really crucial.

But in the "specs are expensive, proofs are cheap" world, we assume some known trusted natural/human-written proof stack, like `Lean` or `Rocq` or `Verus`/`Z3`, is the substrate. What if this assumption is violated?

What if LeoGPT writes a new proof stack, where human interpretability is not a first class feature?

how do we do spec elicitation and validation *then*? how do we govern the proof oracle if we've never seen the syntax before?

TODO: flesh out.

3.17 Formal Theory of Weird Machines in Agents

Stack layers: Execution Harness | Orchestration & Cloud

Blocks: Malicious Model

A weird machine [92] is the unintended computational substrate an attacker programs when individually-legitimate operations compose into capabilities their designers never enumerated. LangSec [90] named this and argued it is the root cause of most exploitation. Von Hippel [94] observes that the framing transfers directly to agentic harnesses: the "gadgets" are the tools the agent is permitted to call (`curl`, `cat`, `man`, a Python sandbox, a vector-store query), and the weird machine is whatever computation the tool set admits under composition. Adversarial agents will live off the land, and "is this permission set safe?" reduces to a question about the reachable states of a composed transition system no one has ever formally described.

Section 3.13 takes this seriously as a widget: specify a capability-safe tool interface in which well-typed compositions cannot reach exfiltration states by construction. That widget ships a type

system and a soundness theorem; what it does not ship is the capability vocabulary the type system needs — which labels matter, what counts as "reachable" across agent boundaries, what threshold separates safe composition from unsafe. That is the theoretical gap this enabler addresses.

The missing theory has two pieces. First, a notion of *capability threshold*: at what point does adding a tool push the composed system past a line that matters? Turing-completeness is the obvious threshold but not quite the right one — a tool set that is Turing-complete over session-local state can still be safe if it cannot reach exfiltration channels, and a sub-Turing set can be lethal if one primitive is a `POST` to an attacker-controlled endpoint. We need a capability-theoretic analogue of complexity classes, indexed by what the tool set can *observably do to the world* rather than what it can compute internally. Second, an inter-agent extension: once agents can message one another, the composition problem is distributed and gadgets cross agent boundaries. The classical weird-machines literature has no distributed analogue.

Von Hippel sketches two paths and picks one. The path he rejects (fn. 8 of [94]) is classical-LangSec-for-agents: cast tool invocations into a correct-by-construction language with inherent safety or runtime-observability guarantees. Meijer's *Guardians of the Agents* [97] is the cleanest current instantiation of that path: it argues that prompt injection is a code-data confusion problem of the same shape as SQL injection, has the LLM emit a structured plan over symbolic references rather than calling tools one at a time, and discharges the plan against a security policy with three independent static checks (taint analysis, security automata, and `Z3`) before any side-effecting tool runs. The path von Hippel prefers leans on cheap proofs to discharge safety obligations per deployment. The second is probably the right long-run bet, but it inherits its soundness from the theory being articulated here: proof oracles need to know what they are proving, and "this permission set does not induce a dangerous weird machine" is not currently a statement anyone can write down rigorously. The correct-by-construction path is still worth understanding, if only because it forces you to name the safety properties the proof oracles will be asked to verify.

This is an enabler rather than a widget because it does not ship a verified artifact. It produces a vocabulary — capability classes, threshold lemmas, decidability results for bounded fragments — that downstream widgets (Section 3.13, Section 3.11) can target. Until that vocabulary exists, those widgets are verifying properties whose significance we can only argue informally.

3.17.1 Solution/project Sketch

Three threads, in rough dependency order.

Capability-class hierarchy. Adapt the existing process-algebra and IFC literature into a taxonomy of tool-set capabilities indexed by observable effects (read, write, exfiltrate, persist, escalate, signal). Formalize "tool set T induces capability C" as a reachability predicate on the composed transition system. Identify fragments where the predicate is decidable (finite data, bounded steps, no unbounded recursion) and fragments where it is not. Deliverable: a `Lean` or `Rocq` library of definitions and a handful of threshold theorems.

Distributed extension. Lift the above to multi-agent settings where agents share a message channel. The relevant analogues are concurrent separation logic and session types — the question becomes whether any protocol reachable from the permitted message grammar induces a capability the

individual agents lack. Expect most interesting cases to be undecidable; the deliverable is a soundly over-approximating type system.

Undecidability boundaries. Prove Rice-style results pinning down where automated analysis of tool-set safety is and is not possible. This matters operationally: it tells harness designers which guarantees they can buy with static checking and which require runtime monitoring or per-session proof-oracle discharge.

The output of all three is papers and libraries, not deployable C code. That is the point of an enabler.

4 Appendix: Adversary taxonomy

The *Invites* tags on each stack layer (Section 2) and the *Blocks* tags on each tractable problem (Section 3) draw from a fixed set of adversary archetypes. The same taxonomy populates the tooltips on the website. Each archetype is named for the position it occupies relative to the target system, not for a specific actor or organization.

Adversary	Description
Supply Chain	Malicious or compromised upstream: poisoned packages (PyPI/npm), tampered model weights, trojaned chips, EDA-tool insertions, counterfeit firmware.
Co-tenant	A workload sharing infrastructure with the victim — same GPU partition, same K8s node, same NVLink fabric — that exploits weak isolation to read or corrupt the victim's state.
Malicious Model	The model or agent itself acting adversarially, whether misaligned, jailbroken, or scheming: escapes its sandbox, composes tool calls into capabilities it was not granted, or emits inputs that exploit downstream parsers.
Rogue Insider	A principal with legitimate access abusing it: stolen signing keys, over-permissioned service accounts, push access to a model registry, BMC takeover.
Network MITM	An on-path attacker between nodes or between client and server: tampering unauthenticated RDMA traffic, compromising an SDN controller, or performing traffic analysis on collective-ops patterns.
Physical Adversary	An attacker with physical access to hardware: probing HBM interposers, EM and power side channels, counterfeit or remarked chips, tampering with server chassis.
External User	An external actor hitting the public API: crafted prompts that trigger deserialization bugs, content-filter evasion, or state-poisoning of rate-limiters and context windows.

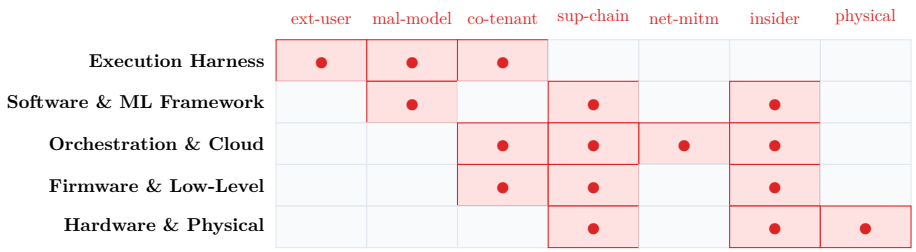

Figure 9: Which adversary archetypes each stack layer invites. The same `// Invites:` headers drive the per-layer tags in Section 2; this matrix is just an at-a-glance view of the bipartite relation.

Bibliography

[1] J. Regehr, "Zero-Degree-of-Freedom LLM Coding using Executable Oracles." [Online]. Available: https://john.regehr.org/writing/zero_dof_programming.html

[2] M. Kleppmann, "Prediction: AI will make formal verification go mainstream." [Online]. Available: https://martin.kleppmann.com/2025/12/08/ai-formal-verification.html

[3] M. von Hippel, "Secure Program Synthesis." [Online]. Available: https://www.lesswrong.com/posts/8wtrLoDPyCfMLuHkt/how-to-solve-secure-program-synthesis

[4] BlueRock Security, "NOVA: A Microhypervisor-Based Secure Virtualization Architecture." [Online]. Available: https://bluerocksec.gitlab.io/formal-methods/faq/what-is-nova/

[5] ggml-org, "GHSA-96jg-mvhq-q7q7: Heap Buffer Overflow via Integer Overflow in GGUF Tensor Parsing." [Online]. Available: https://github.com/ggml-org/llama.cpp/security/advisories/GHSA-96jg-mvhq-q7q7

[6] Trail of Bits, "EleutherAI, Hugging Face Safetensors Library Security Assessment," technical report, May 2023. [Online]. Available: https://github.com/trailofbits/publications/blob/master/reviews/2023-03-eleutherai-huggingface-safetensors-securityreview.pdf

[7] LLVM Project, "CVE-2023-29932 through CVE-2023-29939: Memory-management vulnerabilities in LLVM MLIR." 2023.

[8] Anthropic, "A postmortem of three recent issues." [Online]. Available: https://www.anthropic.com/engineering/a-postmortem-of-three-recent-issues

[9] S. Bhat, A. d. Keizer, C. Hughes, A. Goens, and T. Grosser, "Verifying Peephole Rewriting in SSA Compiler IRs," in *15th International Conference on Interactive Theorem Proving (ITP 2024)*, 2024. doi: 10.4230/LIPIcs.ITP.2024.9.

[10] M. Fehr, Y. Fan, H. Pompougnac, J. Regehr, and T. Grosser, "First-Class Verification Dialects for MLIR," in *Proceedings of the ACM on Programming Languages (PLDI)*, 2025. doi: 10.1145/3729309.

[11] X. Leroy, "Formal verification of a realistic compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009, doi: 10.1145/1538788.1538814.

[12] PyTorch Contributors, "CVE-2025-32434: Remote code execution via \texttt{torch.load} with \texttt{weights_only=True}." 2025.

[13] JFrog Security Research, "Unveiling 3 Zero-Day Vulnerabilities in PickleScan." [Online]. Available: https://jfrog.com/blog/unveiling-3-zero-day-vulnerabilities-in-picklescan/

[14] ReversingLabs, "Malicious ML models discovered on Hugging Face platform." [Online]. Available: https://www.reversinglabs.com/blog/rl-identifies-malware-ml-model-hosted-on-hugging-face

[15] Google / TensorFlow, "CVE-2024-3660: Arbitrary code execution via Keras Lambda layer \texttt{safe_mode} bypass." 2024.

[16] K. Zhang, Y. Feng, and Z. Chen, "Unraveling the Characterization and Propagation of Security Vulnerabilities in TensorFlow-based Deep Learning Software Supply Chain," in *16th Asia-Pacific Symposium on Internetware (Internetware 2025)*, 2025. doi: 10.1145/3755881.3755923.

[17] J. Mahon, C. Hou, and Z. Yao, "PyPitfall: Dependency Chaos and Software Supply Chain Vulnerabilities in Python," 2025.

[18] PyTorch Team, "Compromised PyTorch-nightly dependency chain (torchtriton)." [Online]. Available: https://pytorch.org/blog/compromised-nightly-dependency/

[19] ReversingLabs, "Compromised Ultralytics PyPI package delivers crypto coinminer." [Online]. Available: https://www.reversinglabs.com/blog/compromised-ultralytics-pypi-package-delivers-crypto-coinminer

[20] FutureSearch, "litellm Supply Chain Attack." [Online]. Available: https://futuresearch.ai/blog/litellm-pypi-supply-chain-attack/

[21] SLSA Contributors, "SLSA v1.0: Supply-chain Levels for Software Artifacts." [Online]. Available: https://slsa.dev/spec/v1.0/

[22] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software Signing for Everybody," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022. doi: 10.1145/3548606.3560596.

[23] NVIDIA, "CVE-2024-0132: NVIDIA Container Toolkit TOCTOU container escape." 2024.

[24] Open Containers / Snyk Security Labs, "CVE-2024-21626: runc container escape via leaked file descriptors (Leaky Vessels)." 2024.

[25] SchedMD, "CVE-2023-49935: Slurm MUNGE message integrity bypass." 2023.

[26] SchedMD, "CVE-2022-29501: Slurm PMI2/PMIx handler allows unprivileged write to arbitrary Unix sockets." 2022.

[27] SchedMD, "CVE-2017-15566: Slurm SPANK environment variable privilege escalation." 2017.

[28] Oligo Security, "CVE-2023-48022: Unauthenticated remote code execution in Ray (ShadowRay)." 2023.

[29] Sysdig Threat Research Team, "AI-assisted cloud intrusion achieves admin access in 8 minutes." [Online]. Available: https://sysdig.com/blog/ai-assisted-cloud-intrusion-achieves-admin-access-in-8-minutes

[30] Mithril Security, "PoisonGPT: How we hid a lobotomized LLM on Hugging Face to spread fake news." [Online]. Available: https://blog.mithrilsecurity.io/poisongpt-how-we-hid-a-lobotomized-llm-on-hugging-face-to-spread-fake-news/

[31] GitGuardian, "The State of Secrets Sprawl 2025," technical report, 2025. [Online]. Available: https://www.gitguardian.com/state-of-secrets-sprawl-report-2025

[32] F. Wilhelm, "An EPYC escape: Case-study of a KVM breakout." [Online]. Available: https://projectzero.google/2021/06/an-epyc-escape-case-study-of-kvm.html

[33] Trail of Bits, "CVE-2023-4969: LeftoverLocals — GPU local memory leak across tenant boundaries." 2023.

[34] Y. Zhang, R. Nazaraliyev, S. B. Dutta, A. Marquez, K. Barker, and N. Abu-Ghazaleh, "NVBleed: Covert and Side-Channel Attacks on NVIDIA Multi-GPU Interconnect." [Online]. Available: https://arxiv.org/abs/2503.17847

[35] Advanced Micro Devices, "AMD SEV-TIO: Trusted I/O for Secure Encrypted Virtualization." [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/developer/sev-tio-whitepaper.pdf

[36] NVIDIA, "CVE-2024-0126: NVIDIA GPU Display Driver privilege escalation." 2024.

[37] NVIDIA, "CVE-2024-0107: NVIDIA GPU Display Driver out-of-bounds read." 2024.

[38] A. Zambelli, "Multiple Vulnerabilities Discovered in NVIDIA CUDA Toolkit." [Online]. Available: https://unit42.paloaltonetworks.com/nvidia-cuda-toolkit-vulnerabilities/

[39] NVIDIA, "CVE-2022-21819: NVIDIA Jetson PCIe DMA attack bypassing secure boot." 2022.

[40] G. Stewart, "Device Driver Verification in Separation Logic (talk)." [Online]. Available: https://sel4.systems/Summit/2025/abstracts2025.html

[41] BlueRock Security, "Verifying a Virtual Machine Monitor," Tech report, 2024. [Online]. Available: https://bluerocksec.gitlab.io/formal-methods/tech_reports/verifying-a-virtual-machine-monitor/

[42] BlueRock Security, "Modularizing CPU Semantics for Virtualization," Tech report, 2024. [Online]. Available: https://bluerocksec.gitlab.io/formal-methods/tech_reports/modularizing-cpu-semantics-for-virtualization/

[43] P. Sewell and REMS Group, "REMS: Rigorous Engineering of Mainstream Systems." [Online]. Available: https://www.cl.cam.ac.uk/~pes20/rems/

[44] Microsoft, "CVE-2022-21894: UEFI Secure Boot bypass exploited by BlackLotus bootkit." 2022.

[45] Binarly, "CVE-2023-40284 through CVE-2023-40290: Supermicro BMC IPMI firmware vulnerabilities." 2023.

[46] BleepingComputer, "NVIDIA confirms data was stolen in recent cyberattack." [Online]. Available: https://www.bleepingcomputer.com/news/security/nvidia-confirms-data-was-stolen-in-recent-cyberattack/

[47] A. Classen, D. Hansma, T.-N. Ngo, M. Becker, and K. Rieck, "Evil from Within: Machine Learning Backdoors through Hardware Trojans," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2025. [Online]. Available: https://arxiv.org/abs/2304.08411

[48] K. Basu *et al.*, "CAD-Base: An Attack Vector into the Electronics Supply Chain," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 24, no. 4, pp. 38:1–38:30, 2019, doi: 10.1145/3315574.

[49] J. Harrison, "Formal Verification at Intel," in *18th Annual IEEE Symposium on Logic in Computer Science (LICS)*, 2003, pp. 45–54. doi: 10.1109/LICS.2003.1210044.

[50] A. Reid, "Trustworthy Specifications of ARM v8-A and v8-M System Level Architecture," in *Proceedings of Formal Methods in Computer-Aided Design (FMCAD)*, 2016. doi: 10.1109/FMCAD.2016.7886675.

[51] YosysHQ, "riscv-formal: RISC-V Formal Verification Framework." [Online]. Available: https://github.com/YosysHQ/riscv-formal

[52] OpenHW Group, "CORE-V-VERIF: Functional Verification for CORE-V RISC-V Cores." [Online]. Available: https://github.com/openhwgroup/core-v-verif

[53] J. Choi, M. Vijayaraghavan, B. Sherman, A. Chlipala, and Arvind, "Kami: A Platform for High-Level Parametric Hardware Specification and Its Modular Verification," *Proceedings of the ACM on Programming Languages (ICFP)*, vol. 1, 2017, doi: 10.1145/3110268.

[54] H. Maia, C. Chang, D. Sanchez, and S. Devadas, "Can one hear the shape of a neural network?: Snooping the GPU via magnetic side channel," in *Proceedings of the 31st USENIX Security Symposium*, 2022.

[55] Z. Zhan *et al.*, "Graphics Peeping Unit: Exploiting EM Side-Channel Information of GPUs," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2022. doi: 10.1109/SP46214.2022.9833773.

[56] Semiconductor Industry Association, "Winning the Battle Against Counterfeit Semiconductor Products." [Online]. Available: https://www.semiconductors.org/wp-content/uploads/2018/01/SIA-Anti-Counterfeiting-Whitepaper.pdf

[57] Semiconductor Industry Association and Boston Consulting Group, "Strengthening the Global Semiconductor Supply Chain in an Uncertain Era," technical report, Apr. 2021. [Online]. Available: https://www.semiconductors.org/strengthening-the-global-semiconductor-supply-chain-in-an-uncertain-era/

[58] L. de Moura, "Who Watches the Provers?." [Online]. Available: https://leodemoura.github.io/blog/2026-3-16-who-watches-the-provers/

[59] J. Harrison, "HOL Light." [Online]. Available: https://hol-light.github.io/

[60] M. Adams, "HOL Zero." [Online]. Available: http://www.proof-technologies.com/holzero/

[61] CakeML Project, "Candle: A Self-Compiling Verification of HOL Light." [Online]. Available: https://cakeml.org/candle/

[62] J. Davis, "The Milawa Rewriter and an ACL2 Proof of its Soundness," in *Proceedings of the Seventh International Workshop on the ACL2 Theorem Prover and Its Applications (ACL2 '07)*, Austin, Texas, USA, Oct. 2007. [Online]. Available: https://kookamara.com/jared/milawa/Documentation/Rewriter/rewrite.pdf

[63] K. Claessen and J. Hughes, "QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs," in *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2000, pp. 268–279. doi: 10.1145/351240.351266.

[64] G. Ammons, R. Bodík, and J. R. Larus, "Mining Specifications," in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2002, pp. 4–16. doi: 10.1145/503272.503275.

[65] G. Klein *et al.*, "seL4: Formal Verification of an OS Kernel," in *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, 2009, pp. 207–220.

[66] R. Marchand *et al.*, "Quantifying Frontier LLM Capabilities for Container Sandbox Escape," *arXiv preprint arXiv:2603.02277*, 2026.

[67] UK AI Safety Institute, "AISI Sandboxing Toolkit." 2025.

[68] UK AI Safety Institute, "Docker Sandbox Breakout." 2026.

[69] UK AI Safety Institute, "Inspect AI." 2025.

[70] A. Kaufman, J. Lucassen, T. Tracy, C. Rushing, and A. Bhatt, "BashArena: A Control Setting for Highly Privileged AI Agents," *arXiv preprint arXiv:2512.15688*, 2025.

[71] Q. Dougherty, A. Singh, and M. von Hippel, "BoxArena: An OCI Runtime Escape Benchmark." 2026.

[72] W. R. Bevier, W. A. Hunt, J. S. Moore, and W. D. Young, "An Approach to Systems Verification," *Journal of Automated Reasoning*, vol. 5, no. 4, pp. 411–428, 1989.

[73] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger, "AI Control: Improving Safety Despite Intentional Subversion," in *International Conference on Machine Learning (ICML)*, 2024.

[74] A. Bhatt *et al.*, "Ctrl-Z: Controlling AI Agents via Resampling," *arXiv preprint arXiv:2504.10374*, 2025.

[75] G. C. Necula, "Proof-Carrying Code," in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1997, pp. 106–119. doi: 10.1145/263699.263712.

[76] Q. Dougherty, "Formal Confinement." 2025.

[77] D. Dolev and A. C. Yao, "On the Security of Public Key Protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983, doi: 10.1109/TIT.1983.1056650.

[78] Auth0, "CVE-2015-9235: JWT algorithm confusion in \texttt{jsonwebtoken}." 2015.

[79] B. Blanchet, "An Efficient Cryptographic Protocol Verifier Based on Prolog Rules," in *14th IEEE Computer Security Foundations Workshop (CSFW)*, 2001, pp. 82–96.

[80] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," in *25th International Conference on Computer Aided Verification (CAV)*, in Lecture Notes in Computer Science, vol. 8044. 2013, pp. 696–701. doi: 10.1007/978-3-642-39799-8_48.

[81] Y. Yarom and K. Falkner, "FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack," in *Proceedings of the 23rd USENIX Security Symposium*, 2014.

[82] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-Level Cache Side-Channel Attacks are Practical," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2015. doi: 10.1109/SP.2015.43.

[83] D. Jackson, "Alloy: A Lightweight Object Modelling Notation," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 11, no. 2, pp. 256–290, 2002, doi: 10.1145/505145.505149.

[84] N. Cankaya, A. Friedman, and M. Baker, "Research Note: The Fundamentals and Feasibility of Secure Network Taps for Verifying AI Datacenter Use." [Online]. Available: https://nacicankaya.substack.com/p/research-note-the-fundamentals-and

[85] Amodo Design, "Network Tapping for AI Verification: A Technical Assessment." [Online]. Available: https://amododesign.com/network-tapping-tech-note/

[86] A. Pnueli, M. Siegel, and E. Singerman, "Translation Validation," in *Proceedings of the 4th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 1998, pp. 151–166. doi: 10.1007/BFb0054170.

[87] A. Erbsen, J. Philipoom, J. Gross, R. Sloan, and A. Chlipala, "Simple High-Level Code for Cryptographic Arithmetic – With Proofs, Without Compromises," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2019. doi: 10.1109/SP.2019.00005.

[88] M. Kwiatkowska, G. Norman, and D. Parker, "PRISM 4.0: Verification of Probabilistic Real-Time Systems," in *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV)*, 2011, pp. 585–591. doi: 10.1007/978-3-642-22110-1_47.

[89] C. Dehnert, S. Junges, J.-P. Katoen, and M. Volk, "A Storm is Coming: A Modern Probabilistic Model Checker," *International Journal on Software Tools for Technology Transfer (STTT)*, 2017, doi: 10.1007/s10009-017-0469-y.

[90] L. Sassaman, M. L. Patterson, S. Bratus, M. E. Locasto, and A. Shubina, "Security Applications of Formal Language Theory," 2013, pp. 489–500.

[91] T. Ramananandro *et al.*, "EverParse: Verified Secure Zero-Copy Parsers for Authenticated Message Formats," in *Proceedings of the 28th USENIX Security Symposium*, 2019.

[92] S. Bratus, M. E. Locasto, M. L. Patterson, L. Sassaman, and A. Shubina, "Exploit Programming: From Buffer Overflows to Weird Machines and Theory of Computation," 2011.

[93] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, "Capsicum: Practical Capabilities for UNIX," in *Proceedings of the 19th USENIX Security Symposium*, 2010.

[94] M. von Hippel, "Scalable Formal Oversight: Open Problems in AI Safety and Formal Methods." [Online]. Available: https://www.lesswrong.com/posts/SfhFh9Hfm6JYvzbby/the-scalable-formal-oversight-research-program

[95] A. C. Myers, "JFlow: Practical Mostly-Static Information Flow Control," 1999.

[96] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," *RFC 6962*, 2013.

[97] E. Meijer, "Guardians of the Agents: Formal Verification of AI Workflows," *Communications of the ACM*, vol. 69, no. 1, pp. 46–52, Jan. 2026, doi: 10.1145/3777544.